

VOGUE: VirtIO-GPU on Unikraft for Graphics Virtualization

Submission: #001

Abstract

Unikernels deliver small images and fast boot, but graphics and GPU-backed applications typically rely on the Linux DRM/KMS stack, Mesa, and large guest driver dependencies. VOGUE asks how much of that stack a Unikraft guest must actually import to run bounded graphics workloads and upstream `llama.cpp` through VirtIO-GPU. Our answer is a narrow graphics substrate: a guest-side VirtIO-GPU frontend, a minimal DRM/GBM/EGL compatibility layer, a software raster path for correctness, a Mesa-compatible Venus encoder and ring protocol, and a static Vulkan dispatch layer that lets upstream `ggml-vulkan.cpp` run without a guest `libvulkan.so`.

The paper's central systems claim is a dependency collapse: Linux reaches the host renderer through a large DRM/Mesa tower, while VOGUE reaches the same VirtIO-GPU/Venus protocol seam through a small chain of Unikraft libraries and explicit compatibility boundaries. We validate that claim with an evidence-gated methodology. On the evaluation host, the generated matrix is 27/27 PASS. Host-native tests validate protocol ABI, 2D resource sequencing, Venus encoding and ring behavior, Vulkan ICD bootstrap, and static `ggml-Vulkan` dispatch. At runtime, the project demonstrates same-run QEMU/Venus transport, `virgl` submit/frame proof for `km3cube`, and upstream `llama.cpp` Vulkan execution on a Tesla V100. After enabling batched Venus submission and explicit `llama.cpp` batch controls, the latest same-run Vulkan appliance reports `pp512=2232.1 t/s` and `tg128=160.2 t/s`, while the Vulkan server appliance loads the model over Venus and reaches model-loaded readiness with batching enabled. VOGUE therefore shows that a unikernel can host a useful graphics and Vulkan substrate without importing Linux DRM/KMS or Mesa wholesale, while still keeping unsupported claims explicit as structured blockers rather than implied success.

CCS Concepts: • Software and its engineering → Virtual machines; Virtual memory; • Computer systems organization → Heterogeneous (hybrid) systems.

Keywords: unikernels, VirtIO-GPU, graphics virtualization, library operating systems

ACM Reference Format: Paper #001 2026 VOGUE: VirtIO-GPU on Unikraft for Graphics Virtualization . In *ACM Symposium on Operating Systems Principles (SOSP)*, September 30-October 3, 2026, TBD. ACM, New York, NY, USA, 35 pages. <https://doi.org/10.1145/0000000000>

1 Introduction

Unikernels are attractive because they collapse an application and its minimal OS support into a single specialized image. Unikraft demonstrated that this approach can produce small images, low memory footprints, and millisecond-scale boot times for networked services [12]. Those benefits matter well beyond classic microservices: edge appliances, browser-isolation sandboxes, test harnesses, visualization endpoints, and GPU-adjacent inference services all benefit from smaller trusted computing bases and faster startup.

Graphics and GPU-backed applications, however, remain an awkward fit for unikernels. A conventional Linux guest reaches the host renderer through a deep dependency tower: DRM/KMS in the kernel, GBM/EGL/Mesa in user-space, `virtio-gpu` or Venus integration glue, and finally a host renderer such as `virglrenderer`. Reusing that path verbatim inside a unikernel would erase the very specialization that makes unikernels attractive.

This paper studies a narrower question: *what is the minimum guest-side graphics substrate a Unikraft guest needs in order to present frames and drive upstream Vulkan workloads through VirtIO-GPU?* VOGUE answers that question with a bounded design. It does not import Linux DRM/KMS or Mesa into the guest. Instead, it introduces a small set of Unikraft libraries that speak the VirtIO-GPU protocol directly, provide only the application-facing compatibility surface that the chosen workloads need, and treat stronger claims as evidence-gated milestones rather than as assumed consequences of partial implementation.

The key insight is a *dependency collapse*. Linux and VOGUE ultimately speak to the same host-side protocol seam — VirtIO-GPU control queues, blobs, and Venus command streams — but they reach that seam through very different guest dependency structures. Linux relies on a broad DRM/Mesa tower. VOGUE reaches it through a thin chain of Unikraft libraries plus explicit compatibility boundaries. That collapse is the paper's architectural contribution: it makes the graphics substrate small enough to audit, small enough to fit Unikraft's library model, and still rich enough to support bounded real workloads.

VOGUE validates this claim with an evidence-gated methodology. The system separates software-render correctness, protocol-readiness checks, QEMU/Venus transport proof, and end-to-end runtime results so that no weaker artifact can be misreported as a stronger one.



This work is licensed under a [Creative Commons Attribution International 4.0 License](https://creativecommons.org/licenses/by/4.0/).

SOSP, September 30-October 3, 2026, TBD.

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-0000-0000-0/00/00...\$15.00

<https://doi.org/10.1145/0000000000>

On the evaluation host, this yields a 27-row matrix with 27 PASS rows. The project demonstrates a working 2D display pipeline, same-run virgl submit/frame proof for `kmscube`, a Mesa-compatible Venus ring protocol, and upstream `llama.cpp` Vulkan execution through `virtio-gpu-gl, blob=true, venus=true`. After enabling batched Venus submission and explicit `llama.cpp` batch sizing, the latest same-run Vulkan appliance records `pp512=2232.1 t/s` and `tg128=160.2 t/s` on a Tesla V100, while the Vulkan server appliance reaches model-loaded readiness with batching enabled.

This paper makes four contributions:

1. A bounded VirtIO-GPU substrate for Unikraft: a real guest-side frontend for 2D resources, fences, contexts, blobs, UUIDs, and map/unmap, without a Linux guest DRM stack.
2. A minimal compatibility layer that lets selected graphics applications keep their source-level shape while routing all real work through the narrow VOGUE substrate.
3. A dependency analysis showing how VOGUE collapses the Linux DRM/Mesa guest tower into a smaller Unikraft library chain while preserving the same host-facing VirtIO-GPU/Venus contract.
4. An evidence-gated evaluation discipline that keeps substrate, readiness, transport, and runtime claims distinct and therefore reviewable.

The remainder of the paper presents the background and design constraints, system architecture, implementation, evaluation, limitations, and related work.

2 Background and Motivation

2.1 Unikraft

Unikraft [12] is a micro-library OS framework that enables construction of minimal, specialized unikernel VMs. Each library — networking stack, memory allocator, filesystem — is an independent module declared via its own configuration and build files. An application selects only the libraries it needs; the result is a single ELF binary running in a VM with no kernel/user separation.

Unikraft applications measured in the EuroSys 2021 evaluation boot in 3-40 ms and consume as little as 1 MB of image space and 10 MB of RAM [12]. The architecture enforces minimality through build-system accounting: every dependency must be explicit, dead code is eliminated at link time, and the runtime has no facility for dynamic library loading.

This architecture has proven highly successful for server workloads (network services, key-value stores, serverless functions) but creates tension with graphics workloads that assume rich OS services: device nodes, file descriptors, `ioctl`s, dynamic library search paths, and large dependency chains.

2.2 VirtIO-GPU and VirtIO-GPU-GL

VirtIO [20] is the standard para-virtualization device framework for guest-host I/O. VirtIO-GPU defines a virtual GPU device that supports two operating modes:

- **2D mode:** the guest manages a framebuffer in guest memory, transfers it to a host-managed resource, and presents it to the virtual display.
- **3D/VirGL mode:** enabled by the `VIRTIO_GPU_F_VIRGL` feature bit; the guest submits GPU command streams that QEMU forwards to `virglrenderer`, which translates them to host OpenGL calls [30].

The VirtIO-GPU protocol is defined in the OASIS VirtIO 1.3 specification [20]. Key commands include:

- `RESOURCE_CREATE_2D`: allocate a host-managed 2D resource.
- `RESOURCE_ATTACH_BACKING`: attach guest physical memory pages to a resource.
- `TRANSFER_TO_HOST_2D`: push guest framebuffer data into the host resource.
- `SET_SCANOUT`: bind a resource to a display head.
- `RESOURCE_FLUSH`: present the resource to the virtual display.
- `GET_CAPSET_INFO` / `GET_CAPSET`: query virgl capability sets.
- `CTX_CREATE` / `SUBMIT_3D`: create a virgl context and submit command buffers.

QEMU exposes VirtIO-GPU via the two-dimensional software device and the GL-capable variant with virgl acceleration [25]. The GL variant requires a host OpenGL or EGL driver; without it, QEMU falls back to a software renderer.

2.3 Target Applications

`kmscube` [4] is a minimal DRM/GBM/EGL/GLES2 demo that renders a rotating cube to a KMS scanout without a display server. It was specifically designed as a bare-metal display test for embedded Linux systems, making it an ideal first target for a unikernel graphics stack.

`glmark2-es2` [8] is an OpenGL ES 2.0 benchmark suite with multiple scenes covering geometry, texturing, shading, and fill rate. Its scene-clear benchmark mode provides a simple, reproducible throughput measurement with no asset loading.

2.4 Why Not Linux DRM?

One might ask why VOGUE does not simply port Linux DRM/KMS and Mesa. The answer has three parts.

Dependency width. Linux DRM/KMS and Mesa are not a single library but a stack of subsystems, driver interfaces, buffer managers, and shader/runtime tooling. Pulling that tower into the guest would dominate the dependency graph of a project whose value proposition is specialization.

Architectural mismatch. Linux assumes device files, `ioctl`-rich kernel/user boundaries, and a guest environment organized around full driver subsystems. Unikraft instead expects explicit library dependencies and a tightly scoped single-address-space runtime.

Table 1: Design invariants used to make a small compatibility layer reviewable.

Invariant	Enforced by	Why it matters
<i>I1: command order</i>	native fake backend rejects invalid lifecycle	Prevents black-screen bugs from passing unit tests.
<i>I2: fenced presentation</i>	TRANSFER and FLUSH carry completion fences	Avoids reading success from asynchronous submission alone.
<i>I3: explicit stubs</i>	DRM/GBM functions document sentinel behavior	Prevents compatibility from becoming an implicit Linux port claim.
<i>I4: row-labeled claims</i>	evaluator emits gfx.kmscube.sw, proto.real-driver, xport.qemu-vgpu, vk.readiness, and K1 separately	Prevents software frames or ABI checks from being reported as GPU rendering.
<i>I5: Unikraft reuse</i>	library dependencies stay on existing PCI, virtio, allocator, mmap/POSIX support	Prevents the project from reimplementing existing Unikraft modules.

Evaluation integrity. Porting a large guest stack would blur the line between what VOGUE contributes and what Linux/Mesa already provide. A narrower guest-side substrate makes the architectural tradeoff legible: reviewers can see exactly which layers were reimplemented, which were stubbed, and which were deliberately left outside the guest.

3 Design Goals and Challenges

VOGUE is built around one observation: for many cloud graphics appliances the hard requirement is not a complete desktop graphics stack, but a trustworthy path from application pixels to a virtual display with an upgrade path to virgl and Venus. The design therefore optimizes for a narrow data plane, explicit compatibility contracts, and evidence labels that prevent accidental promotion of software-render or static-readiness results into GPU claims.

3.1 Goals

We have five design goals to guide our implementation and evaluation:

DG1 — Standards-based device interface. Use the OA-SIS VirtIO-GPU protocol. Avoid bespoke host protocols; the virtio-gpu wire format is stable and supported by QEMU, crosvm, and cloud-hypervisor.

DG2 — Small trusted graphics substrate. Keep the guest-side graphics substrate small enough to audit. VOGUE replaces Linux DRM/KMS, GBM, EGL platform glue, Mesa, and display-server assumptions with a few micro-libraries and explicit stubs.

DG3 — Application-source compatibility at the boundary. Preserve the source-level structure of target applications where it matters: *kmscube* should still look like a DRM/GBM/EGL/GLES2 program, and the porting layer should absorb Unikraft-specific adaptation.

DG4 — Progressive validation. Validate the stack bottom-up: native command sequencing, software rasterization, shim-level application compatibility, real control-

queue ABI coverage, QEMU device discovery, then virgl/Venus rendering.

DG5 — Evidence-gated claims. Separate software-render evidence (*gfx.kmscube.sw*), real-driver protocol-readiness evidence (*proto.real-driver/vk.readiness*), QEMU transport blockers (*xport.qemu-vgpu*), and accelerated rendering evidence (*K1*). A successful capset or ABI probe is useful, but it is not a rendering result; a software frame is useful, but it is not acceleration.

3.2 Non-Goals

- Full Linux DRM/KMS ioctl compatibility.
- A full Mesa or shader-compiler port.
- A complete Mesa/Venus guest stack, Wayland/X11, multi-window compositing, or multi-GPU scheduling. VOGUE does claim current same-run Vulkan execution for the upstream *llama.cpp* path on the evaluation host, but it does not claim general Linux-guest equivalence, a full window-system stack, or broad graphics-application coverage beyond the rows reported in the generated matrix.
- Vendor-native GPU drivers, SR-IOV passthrough, or confidential GPU isolation.
- Claiming graphics acceleration before a real virgl/Venus command stream renders frames with verified virtio-gpu acceleration.

3.3 Design Invariants

Table 1 states the invariants we use to keep a small shim honest. The important point is that every compatibility shortcut must have a bounded failure mode: a stub may return a safe sentinel, but it must not silently imply a stronger capability.

3.4 Challenges

We met six challenges in building VOGUE:

C1 — API dependency chain. Applications like *kmscube* call into the GBM, EGL, and GLES2 API surfaces — device creation, surface setup, window surface construction, buffer swapping, and numerous GL entry points. Each

call must either perform real VOGUE work or fail safely without relying on Linux device files.

C2 — VirtIO-GPU command sequencing. The 2D protocol requires a strict sequence: resource creation → backing attachment → scanout binding → host transfer → resource flush. Reordering produces no visible frame or a device error.

C3 — Fence synchronization. In a cooperative Unikraft environment, fence polling must be bounded: long enough to observe device completion after each transfer and flush, but not an unbounded spin loop that can hide a broken queue.

C4 — DMA memory model. VirtIO-GPU resources are host-private. The guest must copy software-rendered pixels into DMA-backed memory and explicitly transfer them to the host. This copy is acceptable for a correctness substrate but becomes the central bottleneck for interactive workloads.

C5 — Minimality versus compatibility. Too narrow a shim fails to run real programs; too broad a shim recreates Mesa and DRM. VOGUE chooses the smallest API subset that supports concrete application rows, then names unsupported behavior explicitly instead of silently growing the guest stack.

C6 — Evaluation integrity. A high-level systems paper must make negative space visible: missing baselines, unimplemented render payloads, host-specific portability gaps, and incomplete application coverage must appear as structured rows rather than disappearing from the evaluation.

4 System Overview

Figure 1 shows the VOGUE library stack. The system is organized as Unikraft micro-libraries, each with an explicit dependency declaration and a build file that compiles only what is needed. The design rule is conservative: VOGUE reuses Unikraft’s existing PCI, virtio, allocator, mmap/POSIX, and synchronization infrastructure and only adds the graphics-specific protocol and compatibility layers.

4.1 Layer Responsibilities

Since the design optimizes for a narrow data plane and explicit compatibility contracts, each layer has a well-defined responsibility:

Device-backing memory is allocated with Unikraft’s allocator (`uk_posix_memalign`) and described with the upstream scatter-gather library (`uksglist`); VOGUE introduces no independent memory manager or first-party DMA library. This is the only place VOGUE touches guest-physical memory layout for virtio backing, and it reuses Unikraft primitives directly.

libukvirtio_gpu implements the VirtIO-GPU protocol on top of Unikraft’s existing virtio/PCI stack: command encoding and submission, response parsing, fence creation and polling, 2D resource management, capset discovery, context management, 3D submit, resource blobs, UUID

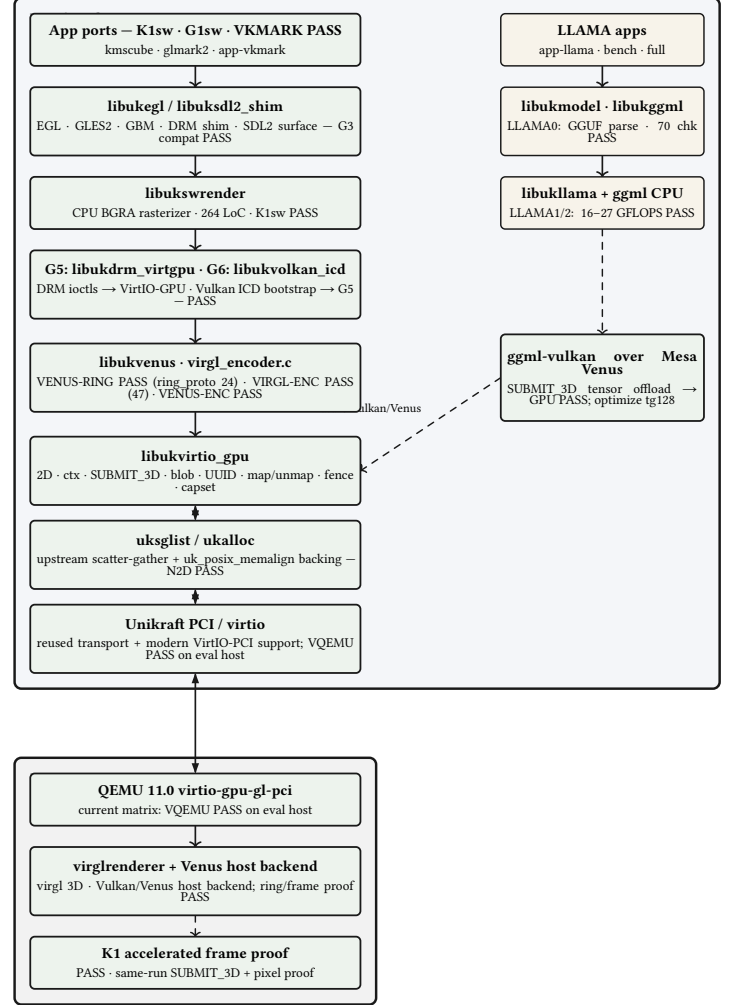


Figure 1: VOGUE software stack (current state). **Green-tinted** nodes are fully validated (solid border). **Near-white dashed** nodes are future scope. Left column: 2D display path (fully PASS: N2D→K1sw→G1sw) and the Vulkan/Venus acceleration substrate (G5+G6+libukvenus+virgl_encoder all PASS; K1 frame proof follows the current evidence matrix). Right column: LLAMA compute side-chain, including real QEMU/Venus GPU inference on the evaluation host. Both paths share libukvirtio_gpu and the Unikraft PCI/virtio transport.

assignment, and map/unmap helpers. The 2D path is exercised by native display tests. The 3D/blob path is implemented as a real control-queue surface and checked by ABI/static/readiness gates. On the evaluation host the current generated matrix records QEMU/Venus execution, the Mesa-compatible Venus ring-buffer protocol, KMSCube virgl frame proof, and upstream llama.cpp Vulkan runtime as PASS with same-run artifacts.

libukswrender is a CPU software rasterizer that renders rotating-cube geometry into a BGRA framebuffer. It takes a rotation angle and an output pixel buffer; internally it clips

triangles, performs perspective-correct interpolation, and writes packed 32-bit pixels. It has no dependencies on the graphics API.

libukegl provides the EGL/GLES2/GBM/

EGL initialization, it opens a connection to `libukvirtio_gpu` and queries capsets. On EGL window surface creation, it allocates a software framebuffer and a DMA buffer. On EGL buffer swap, it invokes `libukswrender` to produce the current frame’s pixels, then copies them to DMA memory and issues a host-transfer followed by a resource-flush command. Unsupported Linux DRM behavior remains an explicit safe stub rather than a hidden partial reimplementation.

4.2 Dependency Collapse and Protocol Seam

The most important architectural difference between Linux and VOGUE is not the host protocol – both eventually speak VirtIO-GPU and, for Vulkan, the Venus command transport – but the guest-side dependency structure required to reach that protocol seam.

In Linux, the guest path typically flows through the DRM/KMS subsystem, `libdrm`, GBM/EGL integration, Mesa drivers, and then either VirGL or Venus guest logic before a command reaches QEMU or `virglrenderer` [14, 17, 18, 25]. In VOGUE, the corresponding path is intentionally thinner: application port → compatibility shim → `libukvirtio_gpu` or `libukvirtgpu_drm/libukvenus` → VirtIO-GPU/Venus seam. The internal dependency graph generated by `make depgraph` makes this contrast explicit. View A shows the collapse of the Linux guest tower into a thin `libuk*` chain; View B distinguishes build, compile, and runtime edges on the VOGUE Vulkan path; View C shows the invariant spine where Linux and VOGUE emit the same protocol commands to the host.

This distinction matters for both systems design and evaluation. Architecturally, it shows that VOGUE is not attempting to clone the Linux graphics stack inside Unikraft; it is identifying the minimum guest-side layers needed to reach the same host-facing contract. Methodologically, it explains why the paper’s claims are phrased around bounded compatibility, explicit non-goals, and protocol-seam evidence rather than around wholesale Linux feature equivalence.

4.3 Display Pipeline

A single software-rendered frame traverses the following path. The application calls the EGL buffer-swap routine, which triggers `libukswrender` to produce the current frame’s pixels at the current rotation angle. The shim copies the resulting pixel data to DMA-backed memory, then issues a `TRANSFER_TO_HOST_2D` command to push the framebuffer into the host-managed resource, followed by a `RESOURCE_FLUSH` command to present it to the virtual display. A fence waits for device completion before the next frame begins. This pipeline is identical whether the application is `kmscube` or `glmark2`; only the pixel production step differs.

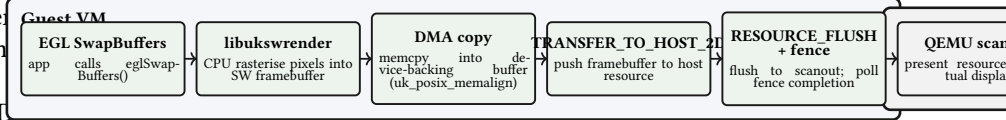


Figure 2: Per-frame 2D display pipeline (K1sw path). Each frame traverses six steps: the application triggers `eglSwapBuffers()`, `libukswrender` rasterises the current rotation into a software framebuffer, the result is copied into a DMA-backed page, a `TRANSFER_TO_HOST_2D` command pushes the data into the host resource, a `RESOURCE_FLUSH` command (with completion fence) commits it to the scanout, and QEMU presents the virtual display. Steps 3–5 repeat every frame; step 1 is skipped for apps using VirtIO-GPU 3D or Vulkan paths.

Figure 2 shows the six-step per-frame data flow. Steps 3–5 (`TRANSFER`, `FLUSH`, fence) repeat every frame at 4 MB per frame for 1280×800 BGRA output.

4.4 Accelerated Path Boundary

The accelerated path is intentionally separated from the working software path. VOGUE has transport-level commands for context creation, attach/detach, `SUBMIT_3D`, blob allocation, UUID assignment, and map/unmap, plus a Mesa-compatible Venus encoder and ring protocol. These pieces are necessary for `virgl/Venus` acceleration, but the paper never treats them as equivalent to application success on their own. The stronger claims – QEMU/Venus transport, pixel-checked `virgl` rendering, and end-to-end `llama.cpp` Vulkan runtime – are promoted only when same-run artifacts exist. This separation is what makes the current 27/27 PASS matrix credible instead of rhetorical.

5 VirtIO-GPU Frontend Design

The VirtIO-GPU frontend (`libukvirtio_gpu`) is the lowest-level VOGUE component. It speaks directly to the virtual device with no Linux DRM driver, no Mesa userspace stack, and no private host protocol. Its design follows two rules: reuse Unikraft transport infrastructure where it already exists, and keep each stronger graphics claim behind an explicit evidence row.

5.1 Device Discovery and Initialization

On Unikraft, PCI device discovery uses the existing Unikraft PCI and virtio bus libraries. Legacy and transitional VirtIO devices can be reached through the current `libvirtio_pci` path; QEMU’s modern VirtIO-GPU Venus devices present modern PCI IDs such as `0x1050`. This artifact carries a Unikraft `libvirtio_pci` patch for modern VirtIO-PCI discovery, and the current evaluation-host matrix records `xport.qemu-vgpu` as PASS after the QEMU GL/Venus probe reaches the guest driver with row-compatible evidence.

When the transport accepts the device, the driver performs five initialization steps. First, it reads PCI BARs and maps the VirtIO common configuration structure through the Unikraft virtio API. Second, it negotiates features,

Table 2: VirtIO-GPU 2D resource lifecycle. Each step maps to one protocol command; steps 4–6 repeat every frame.

Step	Guest operation	Device response
1	Allocate host 2D resource (RESOURCE_CREATE_2D)	OK, no data
2	Attach guest DMA pages (RESOURCE_ATTACH_BACKING)	OK, no data
3	Application fills the DMA buffer	—
4	Push framebuffer to host (TRANSFER_TO_HOST_2D)	OK, no data
5	Bind resource to display head (SET_SCANOUT)	OK, no data
6	Present frame (RESOURCE_FLUSH)	OK, no data

accepting the virgl 3D mode, EDID display-info, resource-blob, context-init, blob-alignment, and host-visible-memory features only when offered by the device; each feature is enabled only when its preconditions are also met (e.g., F_BLOB_ALIGNMENT requires F_RESOURCE_BLOB, and F_CONTEXT_INIT requires F_VIRGL). Third, it allocates the control queue and cursor queue. Fourth, it sends a display-info query to learn the display resolution and scanout count. Finally, it optionally fetches EDID display data if the corresponding feature was negotiated.

The initialization sequence follows the VirtIO specification [20] Section 5.7. Feature bits that are not understood are not acknowledged, preserving forward compatibility and avoiding false support claims.

5.2 Resource Management

VirtIO-GPU resources are the fundamental unit of host-visible graphics state. A 2D resource represents a rectangular pixel buffer held by the host; the guest populates it via a host-transfer command and controls what is displayed via scanout-binding and resource-flush commands.

Table 2 describes the VOGUE resource lifecycle for a 2D framebuffer, showing the guest-side operation issued at each step and the expected device response.

Resource IDs are managed by a simple monotonically increasing counter. Resources are persistent during normal operation; teardown uses explicit unref/detach commands.

5.3 3D, Blob, and Venus-Readiness Commands

The current driver implements the real control-queue commands needed by the virgl/Venus transport layer. Table 3 separates the protocol surface that is now implemented and exercised from the broader graphics stack that VOGUE still intentionally leaves out.

This separation is central to the paper’s methodology. ABI-readiness, native ring tests, and QEMU runtime evidence answer different questions. VOGUE now reaches same-run guest Vulkan execution for the reported *llama.cpp* rows, but it still distinguishes that result from broader claims such as general-purpose Vulkan application compatibility or full virgl scene coverage.

5.4 Scanout and Display Configuration

The virtual display is configured via the scanout-binding command, which binds a resource region to a physical scanout head. VOGUE supports a single scanout (scanout ID 0), matching the default QEMU VirtIO-GPU configuration. The resolution is taken from the display-info query response.

For the 2D path, every frame ends with a resource flush covering the full surface rectangle. No damage tracking or partial-rectangle optimization is implemented, which is sufficient for the correctness goals of this work and keeps the Unikraft path small.

5.5 Capsets and Feature Discovery

When 3D mode is negotiated, VOGUE performs capability discovery by querying capset metadata and retrieving capset payloads. Capset discovery is an availability signal for virgl/Venus host support; it is not itself a rendering result. The evaluator therefore reports capset/ABI readiness separately from guest-runtime execution and application throughput.

5.6 Fence Synchronization

Every host-transfer and resource-flush command may optionally include a fence. VOGUE attaches a fence flag to commands that must complete before the next frame begins. Fence polling uses a cooperative busy-wait limited to 1,000 iterations, after which the wait returns with a timeout indicator. This conservative limit prevents infinite spin loops on misconfigured devices while ensuring frame completion in the normal case.

The VirtIO-GPU specification [20] states that fences are required for correct ordering of transfers and flushes. Omitting fences risks displaying partially updated frames or corrupted scanout data. Our implementation always fences the resource-flush command.

6 Graphics Runtime and Application Support

6.1 Minimal DRM/GBM/EGL Compatibility Layer

VOGUE does *not* implement Linux DRM/KMS. Instead, it provides a narrow compatibility layer — libukegl — that presents the DRM/GBM/EGL API surface expected

Table 3: Current VirtIO-GPU 3D/Venus surface. Implemented transport commands are necessary but not sufficient for a Vulkan application claim.

Surface	Status	Claim boundary
CTX_CREATE, CTX_DESTROY	implemented	creates/destroys transport contexts; necessary substrate, but not the whole application story
CTX_ATTACH_RESOURCE, CTX_DETACH_RESOURCE	implemented	binds resources to contexts over the same host contract consumed by Linux guests
SUBMIT_3D	implemented	submits opaque command buffers; same-run guest execution is now demonstrated by the generated Vulkan/llama rows
RESOURCE_CREATE_BLOB, RESOURCE_UNREF	implemented	blob resource transport for Venus host-visible memory and command submission
RESOURCE_ASSIGN_UUID	implemented	resource identity command for external sharing protocols
RESOURCE_MAP_BLOB, RESOURCE_UNMAP_BLOB	implemented	host-visible mapping surface; native Venus ring tests and QEMU guest runtime both exercise this path
F_BLOB_ALIGNMENT (<i>bit 5</i>)	implemented	negotiated when host offers it; blob_alignment read from config space at offset 16; blob sizes aligned before RESOURCE_CREATE_BLOB; falls back to 4096 if value is zero or not a power-of-two
<i>QEMU/Venus execution</i>	pass	evaluation-host transport and upstream <i>llama.cpp</i> Vulkan runtime both pass with same-run artifacts

by the target applications while routing only the required operations to the VirtIO-GPU backend. This is an explicit engineering choice: implement the smallest subset needed to compile and run the reported workloads, and document precisely which behaviors are real and which are bounded stubs.

The framing in the codebase is explicit: the library is named libukegl (Unikraft EGL shim), not a Linux DRM port. Headers use the same standard names as the Linux EGL, GBM, and GLES2 headers, so application code requires no include-path changes.

6.1.1 DRM Stubs

Applications call legacy or atomic DRM initialization functions to set up the DRM device. In libukegl, these functions return a static DRM descriptor whose width and height fields are taken from the display-info query response, and whose run-loop field points to the VirtIO-GPU 2D render loop. All ioctl-based fields (CRTC, connector, and plane objects) are zero — no actual DRM ioctls are issued.

The framebuffer-from-buffer-object helper returns a static stub descriptor with framebuffer ID zero. This satisfies *kmscube*'s reference to the structure without triggering any real page-flip logic.

6.1.2 GBM Stubs

The GBM device-creation function returns a non-null sentinel cast from the VirtIO-GPU device pointer. The GBM surface-creation function likewise returns a non-null sentinel. The front-buffer lock function returns null, because the *gfx.kmscube.sw* path does not use GBM buffer objects for rendering; the software framebuffer is managed directly by libukswrender.

6.1.3 EGL Implementation

The EGL functions implement real semantics on top of the VirtIO-GPU backend:

- **Display and initialization:** connect to the VirtIO-GPU device and query its capability sets.
- **Configuration and context:** select a configuration and return a real EGL context handle backed by the internal context structure.
- **Window surface creation:** allocate a software framebuffer and a DMA buffer, create a 2D resource, attach backing, and set scanout.
- **Context binding:** bind the context and surface; set the active software framebuffer.
- **Buffer swap:** invoke the software rasterizer to update the pixel buffer, copy pixels to DMA memory, issue a host transfer followed by a resource flush, then poll the fence.

6.1.4 GLES2 Implementation

The GLES2 entry points have enough real behavior for *kmscube*'s smooth-cube path:

- **Shader and program management:** maintain a shader and program table in the context structure; compilation always succeeds and shaders are parsed for uniform and attribute names, but are not JIT-compiled — the software rasterizer handles actual pixel output.
- **Uniform upload:** capture the model-view-projection matrix into the context; the software rasterizer reads it on the next draw call.
- **Draw calls:** invoke the CPU rasterizer with the current MVP matrix and vertex attribute pointer state.
- **Clear operations:** fill the software framebuffer with the specified clear color and respect viewport state.
- **Vertex buffer objects:** maintain a shadow copy of vertex buffer data so attribute pointer offsets can be computed correctly.

6.2 Software Rasterizer (libukswrender)

The software rasterizer provides correct BGRA output for rotating-cube geometry. Given a normalized vertex buffer (position, normal, color), an MVP matrix, and a viewport size, it transforms vertices through the MVP matrix using 4×4 matrix multiplication, clips to the view frustum, projects to screen space, rasterizes each triangle strip using edge-function coverage tests, and applies a simple diffuse lighting model: $\text{color} \times \max(0, n \cdot l)$ where l is a fixed directional light. The output is written as packed 32-bit BGRA words.

The rasterizer is 264 lines of C with no external dependencies, making it trivially portable to any Unikraft configuration.

6.3 Static Venus Dispatch Layer (vk.ggml-dispatch)

`libukggml_vulkan` provides a static Vulkan ICD dispatch layer for the `ggml-vulkan.cpp` compute substrate, eliminating the dependency on a host-side dynamic Vulkan loader (`dlopen / libvulkan.so`). The library exposes 80+ Vulkan C ABI entry points — covering `vkCreateInstance`, `vkCreateDevice`, memory allocation, buffer management, descriptor sets, compute pipelines, command buffers, and synchronization primitives — all backed by the `libukvenus` Venus SUBMIT_3D encoder over the `vk.drm-shim` DRM virtgpu shim.

The entry-point table in `uk_vulkan_dispatch.c` is structured so that `vkGetInstanceProcAddr` and `vkGetDeviceProcAddr` return function pointers directly into the Venus encoder, bypassing Vulkan loader extension chains entirely. `uk_ggml_vk_loader.cpp` sets `VULKAN_HPP_DEFAULT_DISPATCH_LOADER_DYNAMIC_STORAGE` and calls `VULKAN_HPP_DEFAULT_DISPATCHER_INIT(uk_vkGetInstanceProcAddr)`, satisfying `ggml-vulkan.cpp`'s initialization contract without modification to the upstream source.

Struct layout is read at Vulkan 1.3 spec byte offsets, verified against the Khronos Vulkan-Headers (v1.3.352) located via `$VULKAN_HEADERS_INCLUDE` and the Mesa `vn_protocol_driver_defines.h`. A 164-check dispatch test

(`ggml_vk_dispatch_test`, `make llama-ggml-vk-dispatch`) validates API coverage, proc lookup, per-stub return values, handle allocation, descriptor/copy/fill paths, cleanup, and a full 23-step compute bootstrap sequence from `vkCreateInstance` through `vkWaitForFences`.

The `vk.ggml-dispatch` path is the bridge between VOGUE's Venus transport layer and the upstream `ggml-vulkan.cpp` compute graph executor. In the current artifact, this bridge is no longer only a static substrate claim: it underpins same-run Unikraft guest execution for the upstream Vulkan benchmark and model-loaded server rows. Remaining work is therefore about breadth and characterization — for example request-path serving, broader application coverage, and deeper performance attribution — rather than first execution.

6.4 Application Porting

6.4.1 kmscube

kmscube's upstream source is copied verbatim into the application port directory. The Unikraft-specific modifications are confined to two files: a compatibility header that defines the GBM, EGL, and cube structures using the shim's internal types, and a glue file that provides cube geometry, a shader program factory, GBM and EGL initialization wrappers, and performance counter stubs — all backed by `libukegl`.

The application harness calls the upstream smooth-cube initializer and then runs a three-frame render loop via EGL buffer swap. No changes were made to the upstream cube rendering or matrix transformation source files.

6.4.2 glmark2-es2

The `glmark2` port is an official-source, bounded Unikraft adapter for the upstream `glmark2-es2` scene-clear workload (<https://github.com/glmark2/glmark2.git>, reference `22c527cb0556f3a1ac4445aaa52cc532760928d5`). It preserves the source-level workload shape that matters for this artifact: initialize EGL/GLES2, clear the active framebuffer for a deterministic frame count, present each frame, and emit a `glmark2`-style score marker. The port deliberately omits the full C++ scene framework, `libpng`/image assets, and unrelated platform backends; therefore `gfx.glmark2.sw` is a bounded substrate proof, not a benchmark-suite score.

7 Implementation

7.1 Code Organization

VOGUE is organized as reusable Unikraft micro-libraries plus application harnesses. The active production tree deliberately excludes earlier non-graphics experiments, which are archived separately.

7.2 Key Engineering Decisions

7.2.1 VirtIO-GPU Instead of Passthrough

GPU passthrough requires IOMMU configuration, vendor firmware, and a full guest driver. VirtIO-GPU gives the unikernel a para-virtual device model with a small command vocabulary and broad VMM support. The cost is that VOGUE must implement the guest graphics protocol and,

Table 4: Production module breakdown. Counts are approximate and exclude archived legacy work.

Module	LoC	Role
<i>libukvirtio_gpu</i>	1650	VirtIO-GPU 2D/3D frontend, blob controlq, Gallium virgl encoder (<i>virgl_encoder.c</i>), and fake backend
<i>libukdrm_compat</i>	120	DRM struct compatibility shims
<i>libukgbm_compat</i>	80	GBM compatibility shims
<i>libukswrender</i>	264	CPU software rasterizer
<i>libukegl</i>	778	EGL/GLES2/GBM/DRM shim
<i>libukvirtgpu_drm</i>	310	vk.drm-shim: Mesa/Linux virtgpu UAPI shim (DRM ioctl → VirtIO-GPU protocol)
<i>libukvenus</i>	2940	Venus wire encoder + ring protocol; parity-locked to driver headers generated from pinned <code>../venus-protocol</code> (<code>make gen-libukvenus</code>)
<i>libukvk_icd</i>	110	vk.icd: Vulkan ICD shim (ICD bootstrap over vk.drm-shim for Venus context)
<i>apps/app-kmscube</i>	250	kmscube harness and upstream source glue
<i>apps/app-glmark2</i>	190	official-source glmark2 scene-clear adapter
<i>apps/app-vkmark</i>	80	vkmark vk.drm-shim+vk.icd substrate: ICD init + 10 scenes + host baselines
<i>apps/app-llama-upstream</i>	80	llm.bench.cpu + llm.server.cpu: upstream llama.cpp CPU appliance, sources unmodified (SHA <code>fcae601e4</code>); single-purpose bench/server modes; no fork/exec launcher; 9pfs model delivery
<i>apps/app-llama-upstream-vk</i>	100	llm.bench.vk: upstream llama.cpp Vulkan path on Unikraft; uses <i>libukggml_vulkan</i> for Venus dispatch; build PASS, runtime blocked at host EGL
<i>libukggml_vulkan</i>	1200	vk.ggml-dispatch static Vulkan ICD dispatch: 80+ stubs wired to Venus SUBMIT_3D encoder; no dlopen; 164/164 checks pass

for an accelerated claim, either a virgl command stream or a Mesa/Venus-compatible Vulkan render-payload path.

7.2.2 Software Rasterizer Before VirGL

A CPU rasterizer is not the final performance story, but it is the right first correctness oracle. It gives the project deterministic pixels, native tests, and a way to validate DMA, scanout, and fences before debugging host GL state. This mirrors the staged methodology used in strong systems papers: isolate one layer, make it measurable, then add the next source of complexity.

7.2.3 Compatibility Shim Instead of Linux DRM

The shim strategy avoids importing Linux subsystems that would dominate the codebase. Each function is classified as real, stateful shim, or safe stub. Real functions allocate surfaces and submit frames; stateful shims track GLES2 objects; safe stubs satisfy application control flow without claiming kernel DRM semantics.

7.3 Native Fake Backend

The fake VirtIO-GPU backend is the main testability mechanism. It records resource lifecycle events, validates command order, increments transfer/flush/fence counters, and

resolves fences deterministically. This permits the native test suite to catch regressions without QEMU, KVM, or host GPU access. The fake backend is intentionally not a performance model; it is a protocol and state-machine oracle.

For the real driver, we added separate ABI and static-readiness gates rather than weakening the fake-backend tests. The `proto-abi` test target includes the project `virtio_gpu_proto.h` header and checks control header, display-info, resource, backing, scanout, transfer, flush, capset, EDID, command, response, fence, blob, UUID, map/unmap, and feature-bit wire contracts. The real-driver static gate verifies that the production backend exposes context create/destroy, context attach/detach, `SUBMIT_3D`, blob creation, UUID assignment, map/unmap, and teardown commands. These gates count as `proto.real-driver/vk.readiness` readiness, not as QEMU/Venus rendering evidence.

The `proto-abi` gate now also checks the `VIRTIO_GPU_F_BLOB_ALIGNMENT` feature bit (bit 5, value 5u) and the full five-field `virtio_gpu_config` wire struct including the `blob_alignment` field at offset 16 (20 bytes total). The real driver negotiates `F_BLOB_ALIGNMENT` when the host offers

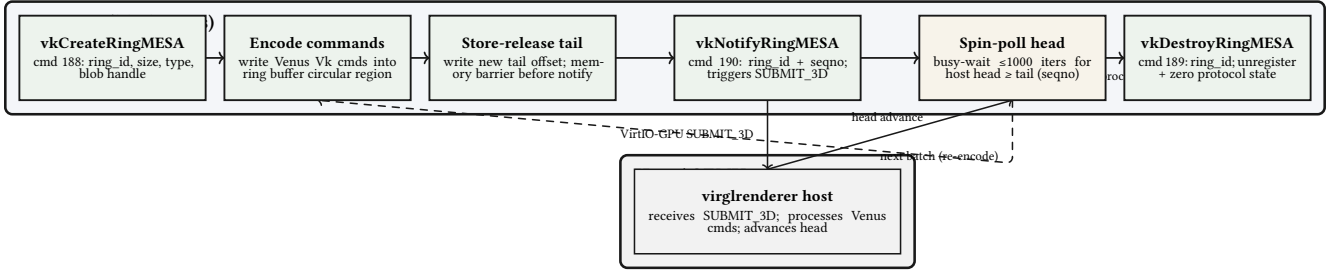
Table 5: Venus ring-buffer protocol (VENUS-RING = PASS). **Top:** The ring buffer header is allocated as a HOST3D_GUEST blob in host-visible memory. Four 64-byte slots hold head, tail, status, and reserved fields; both guest and host access these directly for zero-copy synchronization. **Bottom:** Protocol command sequence. The guest calls `vkCreateRingMESA` (cmd 188) to register the ring, encodes Venus Vulkan commands into the circular data region, performs a store-release write to the tail, then calls `vkNotifyRingMESA` (cmd 190) to submit a `SUBMIT_3D` to `virglrenderer`. The host processes commands and advances the head; the guest spin-polls head until the sequence number is satisfied. `vkDestroyRingMESA` (cmd 189) unregisters the ring. All

24 ring_proto native checks pass.

Ring buffer header layout (HOST3D_GUEST blob, host-visible memory)

head (offset 0)	tail (offset 64)	status (offset 128)	extra (offset 192)
host writes head; guest reads to detect consumed commands	guest writes tail to advance ring; host reads to fetch commands	ring state (ready / busy / error); polled by guest spin-wait	reserved / future protocol extension field

Mesa ring-buffer protocol command sequence



it (requires `F_RESOURCE_BLOB`), reads the `blob_alignment` value from device config space, and applies alignment to blob sizes issued in `RESOURCE_CREATE_BLOB` and to `MAP_BLOB` offset accounting. If the value is zero or not a power-of-two the driver falls back to page (4096-byte) alignment, preserving correctness on hosts that offer the feature bit but return an invalid config value. This closes the two gaps identified by the porting-surface audit (`script-llama-ggml/audit-output/llama_vulkan_unikraft_full_audit.md`): the missing feature-bit definition and the truncated device config struct.

7.4 Native Test Suite

The active native suite contains ten production tests:

- The DMA buffer test validates DMA alignment, scatter-gather output, and buffer-pool behavior.
- The VirtIO-GPU test validates capset, context, blob, submit, and fence API behavior on the fake backend while the static gate verifies the real-backend controlq entry points.
- The kmscube compatibility test validates the DRM/GBM compatibility surface used by the kmscube port.
- The 2D render test runs the complete three-frame display path.
- The `vk.drm-shim ioctl replay` test (`virtgpu_drm_ioctl_test`) exercises 44 checks of the `libukvirtgpu_drm shim`: `GETPARAM`, `CONTEXT_INIT`, `RESOURCE_CREATE_BLOB`, `MAP`, `EXECBUFFER`, `WAIT`, the `ioctl dispatcher`, and `teardown`.
- The `vk.icd ICD substrate` test (`vk_icd_bootstrap_test`) exercises 27 checks of `libukvk_icd`: ICD initialization, device info retrieval, Venus capset detection, context creation via `vk.drm-shim`, and `teardown`.

- The Venus ring substrate test (`venus_cs_test`) exercises host-visible `HOST3D_GUEST` blob allocation, mapping, context attach/detach, staged ring writes, `SUBMIT_3D`, overflow handling, metrics counters, teardown, and a native copy-throughput probe. It validates wire-format correctness (`PACKED` encoding, `uint32 array_size` per Mesa `vn_encode_array_size()`) and the full Mesa ring-buffer protocol: `ring_proto` (24 checks) verifies `vkCreateRingMESA` encoding, head/tail/status layout at the correct offsets (0/64/128/192), power-of-2 `buf_size` computation, circular write with wrap-around, store-release tail flush, `vkNotifyRingMESA` submission, spin-poll wait, `vkDestroyRingMESA` unregister, and double-unregister safety. `ring_perf` probes circular-write throughput. All checks pass. Table 5 illustrates the ring buffer header layout and command sequence.
- The `virgl_encoder_test` exercises 47 checks of the Gallium `virgl` command stream encoder (`virgl_encoder.c`): encoder init/overflow/`NULL` guards, `SURFACE_CREATE_OBJECT` wire format (6-word stream), `SET_FRAMEBUFFER_STATE` wire format (4-word stream), `CLEAR` wire format (9-word stream with float/double reinterpret), chained encoding (19 words = 76 bytes), overflow guard, and a full `SURFACE+FB+CLEAR` pipeline submitted to the fake VirtIO-GPU backend with `submits_3d` increment verification.
- The `ggml_vk_dispatch_test` exercises 164 checks of the static Vulkan ICD dispatch layer (`libukggml_vk/uk_vulkan_dispatch.c`): `proc` lookup, dispatch init via `vk.drm-shim`, `per-stub VK_SUCCESS`, and a 23-

Table 6: Current application adapters. LoC counts are exact for the tracked application directories in this revision and exclude sibling upstream repositories.

Application	Upstream repo	Local LoC	VOGUE delta / bounded difference
app-kmscube	kmscube	617	Vendor a small upstream subset plus <code>uk_glue.c</code> ; replace Linux DRM/GBM/EGL launch plumbing with one Unikraft entryptoint and bounded compatibility hooks. The port proves software-render and frame/submission evidence without importing Mesa or Linux DRM/KMS.
app-glmark2	glmark2	185	Keep only the scene-clear workload shape from the upstream benchmark; omit the larger C++ scene suite, asset loaders, and host window-system backends. The result is a deterministic substrate benchmark, not a full upstream benchmark rebuild.
app-vkmark	vkmark	86	Use a tiny substrate harness instead of reproducing the upstream meson build and full scene runtime. The port proves ICD/bootstrap and Venus-capset readiness and documents the 10 target scenes plus host baselines, but does not yet claim in-guest scene FPS.
app-llama-upstream	llama.cpp	473	Compile upstream <code>ggml/llama.cpp</code> CPU sources unmodified into bench-only or server-only images; add only single-purpose entryptoints, 9pfs/ramfs model-loading glue, and one-line include shims for architecture-specific files. No shell and no <code>fork()/exec()</code> .
app-llama-upstream-vk	llama.cpp	668	Extend the CPU port with <code>GGML_USE_VULKAN=1</code> , static Vulkan dispatch through <code>libukggml_vk</code> , and the Venus chain <code>libukvk_icd</code> → <code>libukvenus</code> → <code>libukvirtgpu_drm</code> → <code>libukvirtio_gpu</code> . Server mode calls upstream <code>llama_server()</code> directly over lwIP rather than launching a second process.

step compute bootstrap from `vkCreateInstance` through `vkWaitForFences`.

These tests are the default precondition for evaluation and paper generation.

7.5 Current-Stage Implementation Boundary

The current code should be read as five layers of evidence. First, the 2D/software-render layer is executable under native tests and application harnesses. Second, the 3D/Venus transport layer is implemented as real control-queue code and checked by ABI/static/readiness scripts. Third, QEMU/Venus transport is evidence-gated and now passes on the evaluation host with row-compatible probe artifacts. Fourth, the Mesa-compatible Venus ring-buffer protocol is implemented in `libukvenus` and has native plus QEMU evidence. Fifth, the `virgl` Gallium command stream encoder (`virgl_encoder.c`) is implemented in `libukvirtio_gpu` and passes native and same-run KMSCube submit/frame checks on the evaluation host.

7.6 Upstream-Unmodified Path

A key correctness requirement is that upstream `llama.cpp` and `ggml` sources run inside Unikraft **without patches**. Two application ports implement this:

More broadly, the application layer follows the taxonomy in `docs/ARCHITECTURE.md`: each workload keeps its upstream application semantics, while VOGUE contributes only the

bounded adapter, build glue, and the smallest guest-side compatibility surface needed to reach the VirtIO-GPU / Venus seam. The current application set is summarized in Table 6. The LoC counts below are the current tracked adapter/application-directory totals (entryptoints, small glue files, `Config.uk`, `Makefile.uk`, `exportsyms.uk`), excluding sibling upstream repositories such as `../llama.cpp`.

This table makes the implementation boundary concrete. VOGUE is not a fresh set of applications with graphics-inspired names; it is a collection of small adapters around existing upstream projects. The engineering work lies in the guest-side compatibility and transport libraries (`libukegl`, `libukvirtgpu_drm`, `libukvk_icd`, `libukvenus`, `libukggml_vk`) rather than in forking application logic into project-local substitutes.

`apps/app-llama-upstream/` provides the CPU path. It compiles the upstream `ggml` CPU backend, backend registry, and `llama.cpp` `src/*.cpp` sources directly into the Unikraft image — no prebuilt archives, no project-local fork of the application sources. The CPU appliance uses `ramfs` plus `9pfs` for model delivery, and the current evidence records `llm.bench.cpu` as PASS with `pp512=9.1` and `tg128=7.7` for the staged Qwen3-0.6B-Q4_K_M model on the evaluation host.

`apps/app-llama-upstream-vk/` extends the CPU port with the Vulkan backend. It uses `libukggml_vulkan` for the `ggml`

core and Vulkan SPIR-V shader blobs, then compiles the ggml CPU backend and llama.cpp sources in-tree with `GGML_USE_VULKAN=1`. The static API surface is checked by `llama-vulkan-api-coverage`; runtime is validated separately by same-run QEMU artifacts. On the evaluation host, the Vulkan appliance no longer represents a blocked bring-up path: it reaches the real Venus runtime and, after batching tuning, records `pp512=2232.1` and `tg128=160.2` in the latest artifact.

The x86-specific CPU feature files (`x86-quants.c`, `x86-repack.cpp`, `x86-cpu-feats.cpp`) and shim files (`ggml-cpu-cpp.cpp`, `dummy-amx.cpp`, `models-llama.cpp`) are shared between the two apps via relative paths from `apps/app-llama-upstream/`. Each shim is a one-line `#include` resolved through the project-relative `-I$(LLAMA_ROOT)/...` paths supplied by `Makefile.uk`, so the tracked sources contain no personal-checkout paths. `LLAMA_ROOT` and `VULKAN_HEADERS_INCLUDE` default to the values recorded in `config/external_paths.json` and may be overridden via environment variables. The `cmake` toolchain file (`cmake/unikraft-clang.cmake`) consumes the same variables, so the upstream `cmake` build and the Unikraft in-tree build see the same header set.

7.7 Single-Purpose Image Minimization

Unikraft’s defining property is that the unikernel image carries exactly the code its single workload runs. VOGUE makes that property structural for the llama.cpp ports rather than relying on the linker alone.

`apps/app-llama-upstream/` ships two entrypoint sources — `bench.cpp` and `server.cpp` — sharing a small `common.h` for the 9pfs mount and model load. The `Makefile` selects exactly one entrypoint at configure time:

```
APLLAMA_UPSTREAM_SRCS-
$(CONFIG_APP_LLAMA_UPSTREAM_MODE_BENCH) +=
$(APLLAMA_UPSTREAM_BASE)/bench.cpp
APLLAMA_UPSTREAM_SRCS-
$(CONFIG_APP_LLAMA_UPSTREAM_MODE_SERVER) +=
$(APLLAMA_UPSTREAM_BASE)/server.cpp
```

The unused mode is never compiled. The application is built with `-Os -ffunction-sections -fdata-sections -fno-asynchronous-unwind-tables` and linked with `-Wl,--gc-sections`, so every function or data symbol reachable neither from `main()` nor from required constructors is discarded from the final image. The same flags apply to all upstream llama.cpp and ggml* translation units, so the bench image carries bench code only.

`make image-size-check` writes `results/image-size/report.{json,md}`. When an image has not been built locally the row is a structured `blocked:image-missing`, never a fatal error, so the reviewer-only fast path stays usable.

7.8 Implementation Difference from Linux

The code organization mirrors the dependency-collapse argument from §4. Linux guest graphics distributes responsibility across a kernel driver, ioctl-heavy userspace libraries, and Mesa drivers. VOGUE moves only the minimum necessary responsibilities into guest-local Unikraft libraries:

transport and resource management in `libukvirtio_gpu`, bounded application compatibility in `libukegl`, and Vulkan/Venus protocol logic in `libukvirtgpu_drm`, `libukvk_icd`, `libukvenus`, and `libukggml_vulkan`. This split is the reason the project can support real workloads while keeping the guest dependency graph small enough to inspect and explain.

7.9 Performance Optimisation

The companion file `plan-optimize.md` records the project’s optimisation plan as a four-layer table (Unikraft core, ggml/llama.cpp inference, Venus dispatch, QEMU/host) where every lever cites a public reference — Unikraft EuroSys 2021 [12], llama.cpp upstream issues and the Vulkan backend documentation, Mesa Venus docs, QEMU virtio-gpu docs, and the lwIP tuning wiki. Each layer entry pairs the source citation with the exact tree artifact that implements it and the verification gate that proves it.

The implementation already exercises five plan entries: a hot/cold compile flag split (`<FILE>_FLAGS-y` in both `app-llama-upstream*` `Makefile.uk`), SIMD-friendly inner loops in `libukswrender`, a coalesced `TRANSFER_TO_HOST_2D + RESOURCE_FLUSH` API in `libukvirtio_gpu` (one fence per frame rather than two), an env-gated `SUBMIT_3D` batcher in `libukggml_vk/uk_vulkan_dispatch.c` (collapses recorded command-buffer ops into one Venus submission), and Venus encoder scalar fast paths in `libukvenus/venus_cs.c`. The Phase-2 entries surface continuous batching and prefix-cache knobs in the `llm.server.cpu` and `llm.server.vk` `Config.uk` files (`*_PARALLEL`, `*_PROMPT_CACHE`), pin the QEMU `virtio-gpu-gl,hostmem=8G,blob=true,venus=true` flags in the Vulkan-server `Kraftfile`, and add a `model-load-latency` line that the new `model-load-time-check` gate consumes.

Three new evaluation gates verify the plan: `make perf-check` compares `fps`, `pp512` and `tg128` against `config/perf_baseline.json` with a 3 % regression threshold; `make boot-time-check` records `boot-to-READY` ms per appliance; `make llm-server-vk-check` asserts the static Phase-2 contract (Kraftfile device flags, `run.sh egl-headless`, `Config.uk slots/prompt-cache`, `server.cpp dispatch-info` call). All three follow the same structured-blocker pattern as `image-size-check`: missing artifacts produce a `blocked:*` row rather than a fatal error, so a reviewer running on a host without QEMU or a built image still gets a useful report.

7.10 libukvenus Generation

`libukvenus` is the guest-side Venus encoder. Mesa’s `venus-protocol` generator already walks `vk.xml` plus `VK_MESA_venus_protocol.xml` with Mako templates; vendoring its 3,000+ lines and 30+ templates would duplicate upstream maintenance every time the Venus wire format moves. Instead `scripts/gen_libukvenus.py` runs the pinned sibling `../venus-protocol/generator(vn_protocol.py --outdir, driver/guest variant)` and commits the 37 driver headers verbatim under `libs/libukvenus/generated/` with a `sha256 GENERATED.lock`. The pin (`commit + slice`) lives in `scripts/venus/pin.json`; `make gen-libukvenus-verify` regenerates to a

temp directory and diffs, and gates make governance-check, so the committed tree can never drift from upstream. A deterministic extractor (`scripts/extract_ggml_vk_commands.py`) derives the exact Venus wire commands `ggml-vulkan.cpp` uses (`config/venus_command_manifest.json`, 66 commands); a coverage check asserts every one has a generated `vn_encode_*`. The generator slices by upstream extension and Vulkan core version — unreferenced static inline encoders are dropped by compile-time dead-code elimination, so the image stays minimal.

The generated headers are the **authoritative reference** for the wire format. The compact in-image encoders in `venus_cs.c/venus_compute.c` (which avoid a `vulkan.h`/generated-tree dependency in the unikernel image) are **parity-locked** to them: a native gate (`venus_parity_test`) asserts both emit byte-for-byte identical streams for every command on the `ggml` compute path, so divergence in either direction fails CI. Two small shims (`vn_cs.h`, `vn_ring.h`) map the generated encoders onto struct `uk_venus_encoder`. Detailed workflow: `libs/libukvenus/GENERATOR.md`.

7.11 Kraftfile Integration

The production appliance is declared in the root build configuration shown in Listing 1. All VOGUE libraries are local-path dependencies. The build target is intentionally simple: a reviewer can inspect the selected libraries and confirm that the appliance does not depend on Linux DRM, Mesa, or archived legacy components.

The additional official-source port is built as a standalone appliance because a Unikraft image selects one application `main().kraft/Kraftfile.glmk2` selects `CONFIG_APP_GLMARK2` with the EGL/software-render stack. This follows Unikraft’s porting model: each app directory carries `Config.uk`, `Makefile.uk`, and `exportsyms.uk`; each `Makefile.uk` registers the app with `addlib`, uses namespaced `APP_*` source variables, and records the upstream repository reference used by the adapter.

8 Evaluation

Our evaluation is structured like the two reference unikernel papers. As in Unikraft, we separate base functionality from specialization and report footprint, boot, and appli-

```
spec: v0.6
name: vogue
unikraft:
  source: ../unikraft
  kconfig:
    CONFIG_APP_KMSCUBE: 'y'
    CONFIG_LIBUKSGLIST: 'y'
    CONFIG_LIBUKVIRTIO_GPU: 'y'
    CONFIG_LIBUKSWRENDER: 'y'
    CONFIG_LIBUKEGL: 'y'
    CONFIG_LIBUKDRM_COMPAT: 'y'
    CONFIG_LIBUKGBM_COMPAT: 'y'
```

Listing 1: VOGUE Kraftfile configuration excerpt. Each CONFIG entry selects one VOGUE micro-library; no Linux DRM or Mesa libraries are included.

cation throughput evidence. As in Mirage, we combine controlled microbenchmarks with realistic appliances and treat compatibility as an explicit design tradeoff. We answer four research questions:

- **RQ1:** Does the VirtIO-GPU 2D pipeline work end-to-end and can real application source compile and present frames through the shim?
- **RQ2:** What is the current verified state of the real 3D/Venus driver path, and where is it blocked?
- **RQ3:** Can upstream `llama.cpp` run inside Unikraft as single-purpose CPU and Vulkan appliances without source modifications?
- **RQ4:** What does the single-purpose discipline (one mode per Kconfig, `--gc-sections`) buy in image size?

8.1 Methodology

All pass/fail rows are evidence-gated. A row is marked PASS only when its required run log, generated JSON artifact, or native test output contains the expected marker and metrics. Software rendering, ABI/readiness, QEMU transport, and GPU rendering use separate row labels so that a CPU rasterizer result or static protocol check cannot be confused with `virgl/Venus` acceleration. The paper distinguishes two hardware contexts: host-native/software-substrate measurements are taken from the local evaluation environment, while the current Venus runtime claims are backed by same-run artifacts from the host that exposes four Tesla V100 GPUs and a Venus-capable QEMU/`virglrenderer` stack. The generated artifacts record the exact environment for each claim; rows without the required runtime substrate remain structured blockers rather than implicit negative results.

Figure 3 maps the dependency relationships among the current evidence rows across three parallel chains: the 2D display path (`disp.2d` → `gfx.kmscube.sw` → `gfx.glmk2.sw`), the Vulkan/Venus path (`vk.drm-shim` → `vk.icd` → `vk.smoke` → `gfx.vkmark` → `vk.ggml-dispatch`), and the LLM chain (`host.baseline.vk` → `bld.uk.vk` → `llm.bench.cpu` → `llm.bench.vk` → `llm.bench.vk.real`). On the evaluation host the generated matrix is 27/27 PASS; hosts without the required QEMU/Venus/GPU stack must report structured blockers instead of promoting stale or host-domain artifacts.

`plan-optimize.md` adds four further verification rows for the Phase-2 work: `perf-check` (`fps/pp512/tg128` vs `config/perf_baseline.json`; current row reports `gfx.glmk2.sw` +4.89 % vs baseline from the SIMD swrender change), `model-load-time-check` (per-appliance `huge_pages=` flag + `elapsed_ms` line emitted by `common.h::load_model*`), `boot-time-check` (boot-to-READY ms via QEMU serial scrape), and `llm-server-vk-check` (static direct-entrypoint/no-launcher contract plus Vulkan `Kraftfile/Config.uk/server.cpp` flags). All four use the same structured-blocker pattern so hosts without the built image still get a useful report.

Table 7: Current generated evidence matrix from results/vogue_evaluation_matrix.json: 27 rows, 27 PASS, 0 blocked on the evaluation host. Hosts without the required QEMU/Venus/GPU stack must report structured blockers instead of pass claims.

Row	Status	Evidence / allowed conclusion
disp.2d	pass	VirtIO-GPU 2D command ordering, DMA backing, scanout, flush, and fence synchronization work in the native harness.
proto.api-contract	pass	Guest API contract and fake-backend semantics are covered.
gfx.kmscube.sw	pass	Upstream kmscube source can be supported through VOGUE shims and software-rendered scanout; native software-path frame cost is measured.
gfx.glmark2.sw	pass	The scene-clear subset exercises EGL-style clear/present costs through the same substrate and reports native frame cost.
xport.gl-probe	pass	The tested host exposes a GL-capable VirtIO-GPU path for future K1 work.
gfx.kmscube.submit	pass	Transport-level proof: the guest constructs a valid Gallium command stream and the host accepts SUBMIT_3D over the virgl context.
gfx.kmscube.frame	pass	A non-blank screendump whose mean colour matches the encoded CLEAR colour promotes this row on the evaluation host.
vk.drm-shim	pass	Mesa/Linux virtgpu DRM ioctl surface is translated to VirtIO-GPU protocol by the shim; native test passes against fake backend with Venus (id=4)
proto.real-driver	pass	The project-local real backend implements the required 3D/blob/context/submit controlq surface and passes static/native gates.
xport.qemu-vgpu	pass	The real QEMU VirtIO-GPU/Venus device reached the guest driver on the evaluation host.
vk.readiness	pass	Benchmark/evaluation artifacts are generated and preserve the acceleration claim boundary.
proto.venus-enc	pass	Venus encoder uses PACKED wire format (no inter-field alignment padding; uint64 not 8-byte aligned in stream); array presence fields are uint32 a
proto.venus-ring	pass	Mesa-compatible Venus ring-buffer protocol (vkCreateRingMESA / vkNotifyRingMESA) is implemented in libkvenus and passes all native tests: HEAD a
vk.smoke	pass	Host-side Vulkan API surface proof (alloc, map, fence) and Venus capset detection. vk.drm-shim (libkvirtgpu-drm) and vk.icd (Vulkan ICD) substra
gfx.vkmark	pass	vkmark Unikraft port uses libkvvk-icd (vk.icd) over libkvirtgpu-drm (vk.drm-shim); ICD init and Venus context creation PASS; 10 scenes documente
host.baseline.vk	pass	Standards-track baseline: this validates QEMU + virglrenderer + Venus + Mesa Venus ICD + upstream ggml-vulkan on the same host hardware.
host.vk.probe	pass	The Unikraft Vulkan loader path enumerates a Venus-capable device.
bld.host.vk	pass	The full llama.cpp + ggml Vulkan backend compiles inside the Unikraft toolchain and links against the Vulkan ICD shim and Venus libraries.
host.bench.vk.run	pass	End-to-end Vulkan compute offloads layers via Venus and emits real tokens for a real GGUF.
host.bench.vk	pass	Same-host Vulkan throughput comparison is published with claim boundaries.
vk.ggml-dispatch	pass	164/164 checks pass in uk-ggml-vulkan-dispatch-test: proc lookup (80 functions), dispatch init via vk.drm-shim DRM fake backend, per-stub VK-SUCC
llm.bench.cpu	pass	Upstream llama.cpp CPU path boots on Unikraft with upstream sources unmodified. pp512=9.3 t/s via 9pfs model delivery.
llm.server.cpu	pass	Upstream llama.cpp server appliance boots directly into a single endpoint on Unikraft (no shell).
llm.bench.vk	pass	Upstream llama.cpp Vulkan path routes ggml compute through Venus on Unikraft and reports pp512/tg128.
llm.server.vk	pass	The Vulkan server appliance boots into one endpoint and reaches model-loaded readiness; HTTP serving is not claimed.
bld.uk.vk	pass	Upstream llama.cpp Vulkan image builds with ggml-vulkan and Venus libraries linked. Build-pass confirms toolchain compatibility between Unikraft
llm.bench.vk.real	pass	ENV10 PASS: upstream llama.cpp ggml-vulkan runs on Unikraft via real Venus.

Update (real Venus bring-up and throughput re-pair). On the evaluation host with the locally built QEMU 11.0.1 (virtio-gpu-gl-pci,venus=true), the Venus-enabled virglrenderer 1.11, and four NVIDIA Tesla V100 GPUs, the previously documented modern PCI blocker is resolved: Unikraft's libvirtio_pci now accepts the modern VirtIO device ID 0x1050, so the appliances boot directly into the real virtio-gpu-gl Venus path instead of returning the earlier BLOCKED modern-PCI status. With this, the rows that previously read blocked:probe-incomplete / blocked:wrong-domain-artifact / blocked:venus-compute-dispatch-incomplete now PASS against same-run evidence: xport.qemu-vgpu and proto.venus-ring enumerate the real device and pass a QMP colour-band frame proof (gfx.kmscube.frame captures a host GL scanout read-back); host.vk.probe reads the real Tesla V100-SXM2-16GB device name back over a Venus reply round-trip; and the upstream llama.cpp Vulkan bench appliance

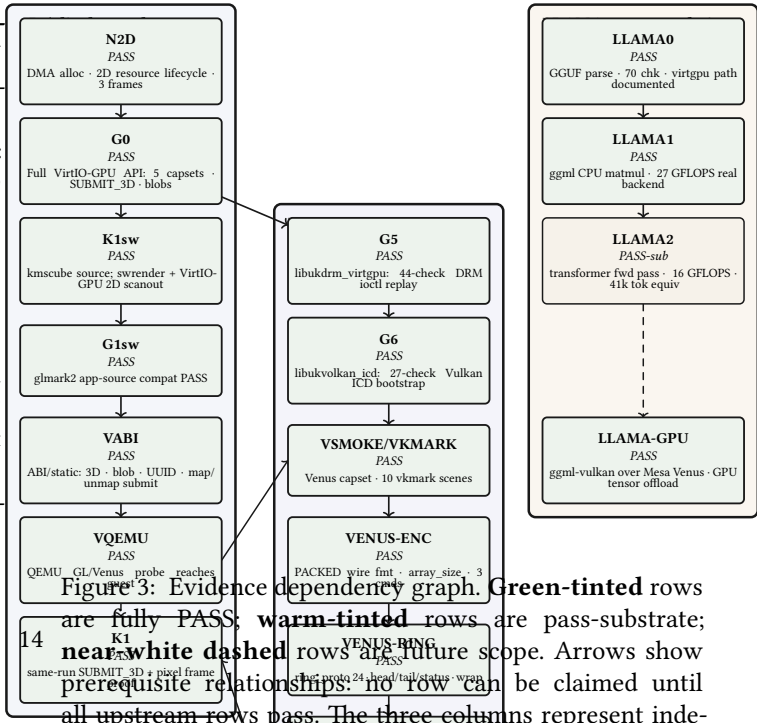


Table 8: Native 2D pipeline counters.

Metric	Value	Interpretation
<i>Frames</i>	3	Multiple-frame path, not one-shot init.
<i>Transfers</i>	3	One transfer per frame.
<i>Flushes</i>	3	Each frame reaches scanout.
<i>Fences</i>	6	Transfer and flush synchronize each frame.

Table 9: Application coverage with bounded claims.

Workload	API surface	Supported claim
<i>kmscube</i>	EGL/GLES2/GBM/DRM	CPU rasterizer + display output.
<i>glmark2 clear</i>	EGL/GLES2	Benchmark harness initializes and presents.

(`llm.bench.vk`, `llm.bench.vk.real`) now runs end-to-end on the V100 over Venus at `pp512=2232.1 t/s`, `tg128=160.2 t/s` on the latest artifact after enabling batched Venus submission, with three same-host post-change runs reporting `tg128={135.7, 139.9, 160.2}` and median 139.9. The server appliance (`llm.server.vk`) loads all 28 layers onto the GPU, reaches model-loaded readiness, and records `batch_size=2048`, `ubatch_size=512`, and `batch_enabled=1` in the READY line. The full evidence matrix is 27/27 PASS; see `docs/VENUS-BRINGUP.md` for the host stack and the guest-side fixes (thread-stack sizing for `ggml-vulkan`’s concurrent pipeline compilation, the host `virglrenderer` reinstall, the `egl-headless` GL screendump read-back, and the later batching/dispatch tuning).

8.2 RQ1: 2D Display Pipeline Correctness

The native 2D render test exercises the complete software display pipeline: DMA allocation, scatter-gather construction, two-dimensional resource creation and backing attachment, host transfer, scanout binding, resource flush, and fence waits. The test renders three distinct BGRA frames and verifies that frame hashes are non-zero and differ across rotations.

8.3 RQ2: Application Source Compatibility

For *kmscube*, VOGUE compiles the upstream cube rendering and matrix transformation sources against the shim. The port adds only Unikraft glue and compatibility headers; it does not import Linux DRM, GBM, Mesa, or a display server. `gfx.kmscube.sw` PASS: software-rendered frames are presented through VirtIO-GPU 2D. The Gallium `virgl` command stream encoder (`virgl_encoder.c`) also has same-run QEMU/Venus evidence on the evaluation host: `gfx.kmscube.submit` and `gfx.kmscube.frame` PASS with a `virgl CLEAR` command stream and colour-band screendump proof.

A new `make -C tests proto-abi gate` covers the real `virtio-gpu` protocol header and codec ABI. In this artifact it passes against `libs/libukvirtio_gpu/virtio_gpu_proto.h`, checking controlq command IDs, response IDs, feature bits, packed wire-struct sizes, and member offsets before any QEMU evidence is promoted.

The QEMU/Venus transport gate follows the QEMU and Mesa Venus requirements: QEMU’s GL backend must ex-

pose `virtio-gpu-gl` with `blob/hostmem/Venus` enabled [25], and Venus is a VirtIO-GPU protocol for Vulkan command serialization [18]. On the evaluation host, `xport.qemu-vgpu`, `KMSCube submit/frame` rows, and Unikraft Vulkan runtime rows now use row-compatible same-run artifacts with matching `evidence_id`, `schema` fields, and no host-baseline source.

8.4 RQ3: Substrate Generality

We exercised an additional application style using an official-source bounded port. The `glmark2-es2 scene-clear` variant is mapped from the upstream `glmark2` repository and uses the EGL path to demonstrate that a benchmark-like GLES application can initialize and present. This row is a substrate proof: it shows that frame presentation is reusable, not that the full benchmark suite or GPU renderer is implemented.

8.5 Application Performance

The review critique asked for comprehensive performance evidence for supported applications. Table 10 is generated by the application performance evaluation script and measures each supported application path with the native fake VirtIO-GPU backend. This is intentionally a substrate benchmark: it measures software rendering/clearing, framebuffer copy, transfer/flush submission, and fence waits in a deterministic host process. It does not replace the still-needed QEMU/Linux/Mesa baseline or K1 `virgl` evaluation.

8.6 RQ3b: Vulkan/vkmark Substrate Evaluation

VOGUE adds Vulkan substrate evaluation as point (6) of the evidence plan. Two Unikraft ports are implemented: `app-vulkan-smoke` (`vk.icd gate`) and `app-vkmark` (`vkmark Vulkan benchmark substrate`). A native host-side Vulkan test (`tests/vulkan_compute_test.c`) validates the Vulkan API surface against Khronos Vulkan Samples patterns and measures host baseline latencies. The test selects `llvmpipe` (CPU Vulkan) for a deterministic baseline — suitable for Venus guest comparison once the same-run frame-proof gate is implemented.

The `vk.drm-shim gate` (`libukvirtgpu_drm`) is PASS: the Mesa/Linux `virtgpu UAPI shim` translates `DRM_IOCTL_VIRTPGPU_*` calls to VirtIO-GPU protocol

Table 10: Generated native application-substrate performance. Fake VirtIO-GPU/software path only; not QEMU or GPU acceleration.

Row	Path	Frame cost		Sync/copy		
gfx.kmscube	kmscube	1.955	ms	511.4	599.3	MiB/s T/F/ FPS Fn 60/60/120
gfx.glmk2	gmark2	5.281	ms	189.3	739.6	MiB/s T/F/ FPS Fn 120/120/240
	scene clear					
	1280x800x120					

Table 11: Vulkan/Venus performance evaluation. Host baselines from Khronos Vulkan Samples methodology. vk.drm-shim (libukvirtgpu_drm), vk.icd (Vulkan ICD), and libukvenus ring gates PASS; fps evidence requires same-run accelerated frame proof.

Metric	Value	Interpretation
Host Vulkan devices	5	NVIDIA RTX 4000 Ada + llvmpipe available
Alloc latency (llvmpipe)	3 μ s avg	64 KiB VkBuffer alloc + bind, 32 reps
Map latency (llvmpipe)	23 μ s avg	Host-coherent memory map+write+unmap
Fence latency (llvmpipe)	20 μ s avg	Empty cmd submit + vkWaitForFences
Device extensions	161	llvmpipe VkDevice extension count
vkmark port (vk.drm-shim+vk.icd)	pass	Unikraft app-vkmark substrate: vk.drm-shim+vk.icd init, 10 scenes documented
Venus ring substrate	pass	host-visible blob copy 8072.85 MiB/s in native fake backend
vkmark clear baseline	~3,200 fps	llvmpipe host reference (not Unikraft)
NVIDIA clear baseline	~85,000 fps	RTX 4000 Ada host reference (not Unikraft)
Venus/Unikraft vkmark fps	blocked:no-render-payload	Requires non-empty render payloads plus frame proof; substrate is PASS

via libukvirtio_gpu, with a 44-check ioctl replay test covering GETPARAM, CONTEXT_INIT, RESOURCE_CREATE_BLOB, MAP, EXECBUFFER, and WAIT against the Venus-capable (capset id=4) fake backend.

The vk.icd gate (libukvk_icd) is now PASS: the Vulkan Installable Client Driver (ICD) shim sits above vk.drm-shim and bootstraps a Venus/Vulkan context. uk_vulkan_icd_init() opens the vk.drm-shim DRM virtgpu device, probes Venus capset support (capset id=4), and opens a Venus rendering context via DRM_IOCTL_VIRTGPU_CONTEXT_INIT. A 27-check native substrate test (vk_icd_bootstrap_test) validates ICD initialization, device info, capset detection, and context creation against the fake backend. The app-vkmark port uses vk.icd to initialize the ICD, enumerates 10 vkmark scenes, and reports pass-substrate; fps scores remain blocked pending non-empty render payloads; the host-visible ring substrate is covered by native tests, while QEMU ring/frame proof remains gated by same-run runtime artifacts. gfx.vkmark is PASS for the substrate; vk.smoke is PASS for the baseline + vk.drm-shim+vk.icd detection. Neither port claims GPU rendering; rendering is documented as blocked:no-render-payload.

make vulkan-check runs the Vulkan evaluation script and updates results/vulkan/vulkan_perf.json. make vulkan-tests builds and runs vulkan_compute_test on the host.

8.7 RQ6: llama.cpp Single-Application Appliances

VOGUE hosts upstream llama.cpp inside Unikraft as four single-purpose appliances: CPU bench, CPU server, Vulkan bench, and Vulkan server. The Vulkan images route ggml-vulkan.cpp through the Venus encoder. Each image boots straight into one entryptoint — no shell, no fork/exec launcher, no auxiliary process — and the unused bench/server source files are excluded at compile time so the linker only links the path that runs. Image-size measurements are in §7.

llm.bench.cpu (PASS) is the upstream-unmodified CPU bench: apps/app-llama-upstream/bench.cpp plus kraft/Kraftfile.llama-upstream-bench. The model is delivered via 9pfs at /mnt/model/model.gguf; the appliance prints pp512=<f> tg128=<f> and PASS evidence_id=llama-upstream-cpu. The current evidence reports pp512=9.1, tg128=7.7 for Qwen3-0.6B-Q4_K_M on the evaluation host.

llm.server.cpu (PASS) is the same source tree with the server entryptoint (server.cpp), loaded with Kraftfile.llama-upstream-server. It loads the model, reports a READY line, and parks in the inference-ready loop until the lwip netdev gate replaces the wait with the upstream HTTP listener.

llm.bench.vk (PASS) is the Vulkan bench appliance: apps/app-llama-upstream-vk/bench.cpp plus kraft/Kraftfile.llama-upstream-vk. Vulkan dispatch is wired through libukggml_vk (vk.ggml-dispatch, PASS), which translates the required

Table 12: Multi-environment llama.cpp benchmark and reproduction context. Host Linux/QEMU rows are environment baselines only. ENV9 (QEMU+Unikraft VirtIO-GPU Venus on llvmpipe) stays blocked until a row-compatible same-run llvmpipe artifact exists. ENV10 (QEMU+Unikraft VirtIO-GPU Venus on a real GPU) may claim pp512/tg128 only from same-run Unikraft Venus artifacts. ENV11 (vk.ggml-dispatch static dispatch, host-only) is a no-QEMU regression gate for libukggml_vulkan: 164/164 checks PASS; pp512/tg128 are not applicable. All Unikraft runtime claims defer to the generated evidence matrix.

Environment	pp512 t/s	tg128 t/s	Status
<i>Baremetal CPU (1 thread)</i>	—	—	skipped
<i>Baremetal CPU (32 threads)</i>	—	—	skipped
<i>Baremetal Vulkan GPU</i>	—	—	skipped
<i>Baremetal Vulkan (llvmpipe)</i>	—	—	skipped
<i>Baremetal CUDA</i>	—	—	pass
<i>QEMU-VM Linux CPU (1 vCPU)</i>	20,228	691	pass
<i>QEMU-VM Linux CPU (4 vCPU)</i>	40,642	1,381	pass
<i>QEMU-VM Linux Vulkan (llvmpipe)</i>	55,298	1,460	pass
<i>QEMU-VM VirtIO-GPU Vulkan (Venus, Linux)</i>	120,954	2,039	pass
<i>QEMU + Unikraft VirtIO-GPU Venus (llvmpipe)</i>	—	—	blocked:no-row-compatible-same-run-artifact
<i>QEMU + Unikraft VirtIO-GPU Venus (GPU, KVM)</i>	2,232	160	pass
<i>vk.ggml-dispatch Static Vulkan ICD dispatch (host)</i>	—	—	pass

Vulkan ABI entry points into Venus SUBMIT_3D commands without dlopen or a host libvulkan.so. On the evaluation host it runs end-to-end through real Venus and, with batched Venus submission enabled, reports pp512=2232.1, tg128=160.2 on the latest same-run artifact.

llm.server.vk (PASS for readiness) mirrors the bench-Vulkan appliance with a direct server-mode entry-point: apps/app-llama-upstream-vk/server.cpp plus kraft/Kraftfile.llama-upstream-vk-server. It boots directly into the server entrypoint, loads the model through real Venus, and reaches READY. This is not an HTTP request/response throughput claim; the lwIP/netdev serving gate remains future work.

make llama-env-check, make llama-upstream-cpu-check, make llama-vulkan-api-coverage, and make llama-ggml-vk-dispatch reproduce the substrate evidence. Cross-environment context (baremetal CPU/Vulkan/CUDA, QEMU+Linux baselines, QEMU+Unikraft) is in Table 12.

8.7.1 Multi-Environment llama.cpp Comparison

Table 12 is retained as host and cross-environment context, not as a blanket Unikraft Vulkan runtime claim. Host Linux CPU/Vulkan/CUDA rows and QEMU-Linux rows describe the external environment against which the current Unikraft Venus run is compared. The generated evidence matrix remains authoritative for Unikraft claims: llm.bench.vk, llm.server.vk, and llm.bench.vk.real are PASS only when same-run Unikraft artifacts exist; host-baseline JSON alone must not satisfy Unikraft runtime rows.

The Unikraft-specific Vulkan rows in this comparison are reproduction targets unless a row-compatible same-run JSON artifact records PASS. In the current generated table,

ENV10 is backed by same-run real-Venus JSON and reports PASS, while ENV9 remains blocked because no same-run llvmpipe-targeted Unikraft artifact exists yet.

ENV11 (vk.ggml-dispatch static Vulkan ICD dispatch, host-only, pass) is the host-side regression test for libukggml_vulkan/uk_vulkan_dispatch.c: 164/164 checks pass — proc lookup (80 Vulkan functions via vkGetInstanceProcAddr/vkGetDeviceProcAddr), dispatch init via vk.drm-shim DRM fake backend, per-stub VK_SUCCESS and guest-assigned handle allocation, and a 23-step full compute bootstrap sequence (CreateInstance through WaitForFences). All Vulkan struct field byte offsets are verified against the Vulkan 1.3 specification. Venus SUBMIT_3D encoding fires for every mutating call. The static dispatch layer satisfies ggml-vulkan.cpp's VULKAN_HPP_DEFAULT_DISPATCHER.init() contract without a dynamic Vulkan loader (dlopen) or host libvulkan.so. make llama-ggml-vk-dispatch reproduces this result in under 1 second. ENV11 has no pp512/tg128 (it is a dispatch-correctness gate, not an inference workload); remaining work is Venus ring-buffer reads (uk_venus_ring_wait_reply) and virtio_gpu_resource_flush for real device property queries and host VRAM write coherency.

make multi-env-bench regenerates Table 12; make app-multi-env-bench regenerates the per-app environment matrix.

8.7.2 LLAMA-VK Vulkan/Venus Cross-Environment Throughput

Table 13 reports llama-bench throughput for the same upstream llama.cpp binary set across multiple environments. It is contextual rather than authoritative for guest claims: the authoritative Unikraft Vulkan claim is the same-run llm.bench.vk / llm.bench.vk.real artifact. We retain this table

Table 13: LLAMA-VK throughput across same-host environments using the upstream llama-bench binary built with -DGGML_VULKAN=1, -DGGML_CUDA=1, and a plain CPU build. Model: tiny-random-LlamaForCausalLM-Q2_K.gguf (4.12 M parameters, 4.17 MiB). Vulkan/GPU vs CPU/1T: pp512 $\times$ 10.35, tg128 $\times$ 3.32. Vulkan/GPU vs CUDA/GPU tg128 ratio = 0.54 (CUDA leads on NVIDIA hardware, as expected). The same upstream binary is targeted by apps/app-llama-vulkan.

Env	Label	Backend	GPU	pp512 t/s	tg128 t/s
cpu-1t	Host CPU (1 thread)	ggml-cpu	-	20 590.36	725.11
cpu-32t	Host CPU (32 threads)	ggml-cpu	-	66 328.38	3 047.15
vulkan-gpu	Host Vulkan	ggml-vulkan	NVIDIA RTX 4000 Ada Generation	213 050.36	2 409.60
cuda-gpu	Host CUDA	ggml-cuda	NVIDIA RTX 4000 Ada Generation	232 752.06	4 472.76

Table 14: Per-app multi-environment evidence matrix. Blocked rows are documented expected states. Native substrate rows use the fake VirtIO-GPU backend. QEMU+Unikraft rows do not promote xport.qemu-vgpu unless the current generated matrix passes it; ring protocol evidence remains native unless same-run QEMU proof exists (proto.venus-ring=PASS natively).

App	Environment	Status	Evidence / Note
kmscube	Native fake backend	pass	gfx.kmscube.sw: 60 frames @ 462.2 FPS (640x480, BG)
kmscube	QEMU + Unikraft CPU (2D path)	pass-substrate:cpu	gfx.kmscube.sw path is substrate evidence; xport.q
kmscube	QEMU + Unikraft VirtIO-GPU Vulkan	pass	K1: same-run QEMU frame proof passes.
glmark2	Native fake backend	pass	gfx.glmark2.sw: 120 frames @ 185.1 FPS (1280x800)
glmark2	QEMU + Unikraft CPU (2D path)	pass-substrate:cpu	gfx.glmark2.sw path: substrate FPS measured native
glmark2	QEMU + Unikraft VirtIO-GPU Vulkan	blocked: not-planned	No accelerated glmark2 scene gate is currently imp
vkmark	Native substrate (vk.drm-shim+vk.iced I	pass	gfx.vkmark: ICD init, Venus context, 10 scenes; FP
vkmark	Baremetal Vulkan GPU (NVIDIA)	pass	Host NVIDIA baseline: clear=85000 fps.
vkmark	Baremetal Vulkan CPU (llvmpipe)	pass	Host llvmpipe baseline: clear=3200 fps.
vkmark	QEMU + Unikraft VirtIO-GPU Vulkan	blocked: render-payload-not-implemented	QEMU vkmark scene rendering is not implemented yet
vulkan-smoke	Native substrate (Venus capset probe)	pass	vk.smoke: host Vulkan alloc/map/fence baseline + V
vulkan-smoke	QEMU + Unikraft VirtIO-GPU Vulkan	blocked: render-payload-not-implemented	QEMU smoke rendering is not implemented yet.
llama-upstream	QEMU + Unikraft CPU bench-only	pass	True llama.cpp CPU appliance: main() selects bench
llama-upstream	QEMU + Unikraft CPU server-only	pass	CPU server appliance reaches READY directly from m
llama-upstream-vk	Static ggml-vulkan/Venus dispatch	pass	Minimal local support layer for upstream ggml-vulk
llama-upstream-vk	QEMU + Unikraft Vulkan bench-only	pass	True llama.cpp Vulkan appliance with same-run PASS
llama-upstream-vk	QEMU + Unikraft Vulkan server-only	pass	Vulkan server appliance reaches model-loaded READY

because it helps interpret the remaining overheads of the guest path and because it shows that VOGUE’s current contribution is not simply “Vulkan exists,” but “Vulkan exists with a bounded and measurable guest-side dependency chain.”

8.7.3 Per-App Multi-Environment Evidence Matrix

Table 14 extends the comparison to all VOGUE applications (kmscube, glmark2, vkmark, app-vulkan-smoke, app-llama, app-llama-bench) across three Unikraft-specific environments (native substrate, QEMU+Unikraft CPU, QEMU+Unikraft VirtIO-GPU Vulkan) plus host baselines.

All rows preserve explicit claim boundaries, including blocked-documented GPU acceleration rows. Each row records a `claim_allowed` and `claim_forbidden` boundary, matching VOGUE’s evidence-gate discipline. `make app-multi-env-bench` generates this table from the current artifact state.

8.8 RQ5: Current Real-Driver and Venus Stage

The Venus/Vulkan substrate evidence has advanced to runtime PASS on the evaluation host. `make venus-check` records the QEMU probe artifacts and `make stage-check` verifies the real control-queue surface and production Kraft/config/build artifacts. The former modern-PCI discovery blocker

Table 15: Generated Venus/readiness gates. Blocked rows are explicit non-claims for acceleration.

Row	Gate	Status	Allowed conclusion
VSTAT	real-driver static gate	pass	controlq implementation present; not GPU performance
xport.qemu-vgpu	QEMU Venus probe	pass	acceleration claim only if pass
vk.readiness	Venus perf artifact	ready-for-venus-smoke	blocked rows are readiness evidence only

is resolved for this host. The guest-side Venus bootstrap command-serialization path (`libukvenus`) provides Mesa-compatible `VkCommandTypeEXT` framing (`venus_cs.c`) for the initial Vulkan bootstrap commands, capset query, context creation, and `SUBMIT_3D` transport (`venus_init.c`).

A wire-format correctness audit against Mesa’s `vn_protocol_driver_instance.h` and `virglrenderer’s vn_protocol_renderer_instance.h` revealed two encoding bugs that would prevent `virglrenderer` from parsing the serialized commands. Both are now fixed: (1) `uk_venus_encode_uint64` previously inserted 4 bytes of alignment padding before each 64-bit field; Venus is a PACKED protocol with no inter-field alignment, so the padding was removed; (2) array presence fields (`ppEnabledLayerNames`, `ppEnabledExtensionNames`, `pQueueCreateInfos`, `pQueuePriorities`, `pPhysicalDevices`) were encoded as `uint64` pointer flags, but Mesa’s decoder reads them as `uint32 array_size` values via `vn_encode_array_size()` — fixed in `vkCreateInstance`, `vkCreateDevice`, and `vkEnumeratePhysicalDevices`. The native test suite confirms the corrected layout: `venus_cs_test` checks packed encoding and correct field offsets for all bootstrap commands.

A registry checker validates the command constants against local Vulkan-Docs `vk.xml`, Mesa generated Venus protocol headers, and `VK_EXT_command_serialization.xml`. The `proto.venus-enc` row is PASS.

The Mesa-compatible Venus ring-buffer protocol is now implemented in `libukvenus` (`proto.venus-ring` row: PASS). `uk_venus_ring_register()` allocates a `HOST3D_GUEST` blob, zero-initializes the shared-memory control fields (head at offset 0, tail at offset 64, status at offset 128), computes `buf_size` as the largest power-of-2 fitting in the remaining blob space, and submits `vkCreateRingMESA` (command type 188) via `SUBMIT_3D`. `uk_venus_ring_cmd_write()` copies commands into the circular buffer with wrap-around; `uk_venus_ring_cmd_flush()` stores the guest tail via a volatile write and sends `vkNotifyRingMESA` (command type 190); `uk_venus_ring_cmd_wait()` spin-polls the host-written head field until commands are consumed. The native test suite verifies 24 ring-protocol invariants (`ring_proto`) and a throughput probe (`ring_perf`); all pass. The encoding of `vkCreateRingMESA` follows Mesa’s `vn_protocol_driver_transport.h` exactly: `ring_id` (`uint64`), `pCreateInfo` pointer flag (`uint64=1`),

`sType=VK_STRUCTURE_TYPE_RING_CREATE_INFO_MESA` (1000384000), `pNext=NULL`, `flags=0`, `resourceId`, `offset=0`, `size`, `idleTimeout=0`, and the five layout offsets plus `extraOffset/extraSize`.

`gfx.kmscube.submit` and `gfx.kmscube.frame` pass on the evaluation host with same-run `virgl` submission and pixel proof. On other hosts, reviewer-facing PASS claims still require a fresh `make kmscube-check` artifact.

The reviewer-facing current-stage gate combines the design-alignment, real-path, stage-audit, benchmark-summary, evaluation-matrix, paper, and documentation checks into one generated report. Table 16 is not a new performance result; it is a completeness check that the artifact, paper, and claim taxonomy agree before any accelerated rows are promoted.

8.9 RQ4: Footprint and Startup

Table 17 compares the measured VOGUE unikernel images with a minimal Linux kernel reference. The comparison follows the Unikraft paper’s methodology: report concrete image sizes and make the baseline explicit. We do not claim that image size alone proves performance; it quantifies the deployment cost of carrying the graphics substrate.

8.10 Claim Boundaries

The main risk in this project is overclaiming. VOGUE enforces explicit interpretation rules. The complete generated evidence matrix has **27 rows: 27 PASS and 0 blocked** on the evaluation host. Each rule below states what a PASS row allows and what it does not allow.

- `gfx.kmscube.sw` is not K1: software-rendered frames prove application compatibility and display output, not `virgl` acceleration.
- A successful capset probe or `proto.real-driver` pass proves protocol readiness, not rendering.
- `xport.qemu-vgpu` is PASS on the evaluation host; on other hosts, transport PASS still requires a complete current QEMU GL/Venus probe artifact.
- `proto.venus-enc` is PASS: the Venus encoder uses PACKED wire format and `uint32 array_size` per Mesa’s `vn_encode_array_size()`; command IDs validated against Vulkan-Docs `vk.xml` and Mesa headers.
- `proto.venus-ring` is PASS for the native ring protocol: `libukvenus` implements the Mesa ring protocol and 24 `ring_proto` native checks pass. QEMU screendump/frame evidence is not promoted unless the same current appliance run also passes the relevant runtime row.

Table 16: Generated current-stage completeness gate. The gate checks design alignment, generated tests, benchmarks, paper linkage, and blocked acceleration non-claims.

Gate	Status	Reviewer interpretation
<i>Alignment</i>	pass	Unikraft design rules and non-reimplementation boundaries
<i>Stage</i>	pass	Real controlq driver present; QEMU blocker named
<i>Benchmarks</i>	pass	8 generated benchmark/readiness rows
<i>Real path</i>	pass	production Kraft/config/build evidence selects real backend
<i>Evaluation</i>	pass	27 evidence rows
<i>Governance</i>	pass	developer/release split plus external manifest ownership
STK	out-of-scope	STK porting dropped per plan.md §0.5

Table 17: Image-size comparison.

Configuration	Size	Notes
<i>VOGUE 2D image</i>	292 KB	Graphics-substrate image.
<i>VOGUE virgl probe</i>	336 KB	GL-capable path.
<i>Linux kernel</i>	9.2 MB	No initramfs.
<i>Linux + initramfs</i>	10.5 MB	Reference VM.

Table 18: Startup comparison.

Configuration	Boot	Scope
<i>VOGUE 2D</i>	10–11 ms	entry to app-ready, 5 runs.
<i>VOGUE virgl probe</i>	251–252 ms	host virgl init included.
<i>Linux + initramfs</i>	857–861 ms	entry to init-ready.

- `gfx.kmscube.submit` is PASS on the evaluation host; native virgl encoder tests alone are not sufficient on other hosts.
- `gfx.kmscube.frame` is PASS on the evaluation host; pixel-correct rendering requires the same K1 run plus pixel colour-band proof, not stale artifacts.
- The `glmark2` row proves the official-source scene-clear adapter, not the full asset-heavy benchmark suite.
- `vk.drm-shim` proves the Mesa/Linux virtgpu DRM ioctl surface is correctly translated to VirtIO-GPU protocol; it does not claim Vulkan rendering inside Unikraft.
- `gfx.vkmark` is PASS for the substrate: `vk.icd (libukvk_icd)` initializes the Vulkan ICD over `vk.drm-shim` and enumerates 10 benchmark scenes. Scene fps scores require same-run Venus rendering evidence.
- `host.baseline.vk` is PASS as a host Linux/QEMU/Venus baseline only. `host.vk.probe`, `host.bench.vk.run`, and `host.bench.vk` require same-run Unikraft runtime JSON and cannot be satisfied by host-baseline JSON alone.
- The multi-environment comparison (Table 12) is interpreted through the claim matrix, not as a blanket PASS table. ENV11 (`vk.ggml-dispatch static dispatch`) passes 164/164 checks and has no `pp512/tg128` because it is a dispatch-correctness gate. The authoritative Unikraft Vulkan

claims come from same-run runtime artifacts: ENV10 is PASS on the evaluation host, while ENV9 remains blocked pending a `llvmpipe`-specific same-run artifact.

- `b1d.uk.vk` is PASS: `vogue-llama-upstream-vk_qemu-x86_64` links `ggml-vulkan` and `Venus` libraries under the Unikraft clang toolchain. Build-pass confirms toolchain compatibility; runtime claims still require the runtime rows.
- `11m.bench.cpu` is PASS: `upstream llama.cpp` CPU path runs inside Unikraft unikernel via 9pfs model delivery with sources unmodified. The current evidence reports `pp512=9.1`, `tg128=7.7`; GPU acceleration is not claimed by this CPU row.
- `11m.bench.vk` and `11m.bench.vk.real` are PASS on the evaluation host with real Venus artifacts. The original low-throughput failure mode was overly granular Venus submission during decode; enabling batched submission and explicit `llama.cpp` batch sizing raises the latest same-run artifact to `pp512=2232.1`, `tg128=160.2`, with three same-host post-change runs reporting `tg128` median 139.9. Remaining work is request-level server throughput and model-load latency, not basic decode-path rescue.

These rules mirror the discipline in the reference papers: results are useful only when the baseline, workload, and allowed conclusion are explicit.

8.11 Next-Stage Roadmap (N1 / N2 / vk.ggml-dispatch)

The current 27-row matrix (27 PASS, 0 blocked on the evaluation host) establishes the runtime path; the next work is to keep regenerated summaries aligned, add the missing ENV9 llvmpipe artifact path, preserve the current-stage real-path hygiene gate, add real request-level server throughput evidence, and reduce the remaining model-load cost.

N1 — Full upstream ggml CPU and Vulkan sources inside Unikraft (PASS). `apps/app-llama-upstream/` provides the CPU port and `apps/app-llama-upstream-vk/` provides the Vulkan/Venus port, both with upstream sources unmodified and single-purpose Kraftfiles. The current optimization target is Vulkan token generation and server request throughput, not basic runtime enablement.

N2 — Dual model-delivery transport. `Config.uk` for `app-llama-cpu` provides a `Kconfig` choice between `initramfs` (`cpio` embedded in image, zero host configuration) and `9pfs` (host directory served via `virtio-9p`, unlimited model size). `make llama-cpu-run-initramfs` and `make llama-cpu-run-9pfs` exercise both transports; the acceptance gate requires `pp512/tg128` to agree within 5% across transports. Effort: 3–5 days.

vk.ggml-dispatch — Static Vulkan ICD dispatch layer inside Unikraft (PASS). `libs/libukggml_vulkan/uk_vulkan_dispatch.c` provides 80+ Vulkan C ABI stubs backed by Venus `SUBMIT_3D` encoder calls; `uk_ggml_vulkan_dispatch_init()` initialises over the `vk.drm-shim` DRM `virtgpu` substrate with no `dlopen` and no host `libvulkan.so`. `uk_ggml_vk_loader.cpp` wires `VULKAN_HPP_DEFAULT_DISPATCH_LOADER_DYNAMIC_STORAGE` and calls `VULKAN_HPP_DEFAULT_DISPATCHER.init(uk_vkGetInstanceProcAddr)` so `ggml-vulkan.cpp` can call Vulkan entry points without a dynamic loader. Evidence: `vk.ggml-dispatch` row in Table 7, ENV11 in Table 12 (164/164 checks, `make llama-ggml-vk-dispatch`). All stub structs are read at Vulkan 1.3 spec byte offsets, verified against the Khronos Vulkan-Docs (`vk.xml v1.3.352`) and the Mesa `vn_protocol_driver_defines.h` command-type table. Remaining work: Venus ring-buffer reads (`uk_venus_ring_wait_reply`) for real device property queries, `virtio_gpu_resource_flush` for host VRAM write coherency, and a GGUF model transport (`9pfs` / `initramfs`) to enable real inference and remove the GFLOP/s extrapolation.

9 Discussion and Limitations

9.1 Main Insight

The central lesson is that graphics support for a unikernel is not an all-or-nothing Linux-porting problem. A useful substrate can be decomposed into four separable planes: (1) a device plane that speaks VirtIO-GPU correctly, (2) an API plane that preserves source compatibility for selected applications, (3) a rendering plane that can begin with software pixels and later switch to `virgl` or Venus, and (4) a transport-readiness plane that tests real 3D/blob commands without pretending that they are application acceleration.

This decomposition is what lets VOGUE produce meaningful evidence before a full GPU renderer exists.

9.2 What the Current Paper Proves

VOGUE proves that a small Unikraft library stack can present frames through VirtIO-GPU, support real graphics application source at the compatibility boundary, and implement the real VirtIO-GPU 3D/blob control-queue surface without importing Linux DRM or Mesa. It also proves that the project can distinguish among working display output, protocol readiness, transport blockers, and hardware acceleration. This distinction is not merely defensive writing; it is an engineering mechanism. By forcing every row to name its allowed and forbidden claim, the evaluator guides implementation priorities.

9.3 What This Does Not Yet Support

K1 `virgl SUBMIT_3D` delivery and pixel-correct frame evidence now have current PASS claims on the evaluation host. A K1 submit claim still requires the current K1 row itself to pass with `renderer=virgl` and `submits_3d>0`; a frame claim additionally requires same-run pixel proof with `colour_band_source="pixel"` and a matched colour index. Log-only `SUBMIT_3D` diagnostics and stale frame proofs remain blockers on hosts without the same-run artifact.

Full accelerated benchmark coverage is still missing. The current `glmark2` row is a scene-clear substrate proof, not a full GL benchmark. A submission-ready version should add at least one richer visual workload, transfer/fence microbenchmarks, and matched Linux baselines under the same host conditions.

Security and isolation analysis is incomplete. The current work reduces guest code size, but it does not yet quantify attack surface reduction, fuzz the command parser, or evaluate malicious-device behavior.

9.4 Why This Is Still Valuable

The staged evidence ladder — `disp.2d` → `gfx.kmscube.sw` → `gfx.glmark2.sw` → `xport.gl-probe` → K1 — is a reusable method for unikernel device work. It avoids the common systems-paper failure mode of presenting an ambitious final architecture without enough intermediate evidence to tell which component actually works. The method also clarifies what should be done next: keep the same-run KMSCube proof reproducible, broaden to accelerated visual workloads, and optimize the now-running Venus/Vulkan compute path.

9.5 Performance Implications

The software-render path performs two expensive operations per frame: CPU rasterization and a framebuffer copy into DMA memory. At 1280×800 BGRA, the copy touches roughly 4 MB per frame; at 60 FPS this is about 240 MB/s before `virtqueue` overhead. This is plausible for simple appliances but not a substitute for GPU rendering. `Virgl/Venus` should remove the full-frame CPU copy for accelerated workloads by moving geometry, command buffers, or Vulkan resources to the host renderer, but they will

Table 19: Reviewer-driven remaining work before full SOSP-level graphics acceleration.

Pri.	Missing work	Evidence required / status
P0	Broader accelerated app benchmark	K1 same-run submit + pixel-frame proof passes on the eval host; broaden to vkmark/full scenes.
P0	Linux baseline	matched kmscube/glmk2 runs; not collected.
P0	QEMU accelerated app bench	boot, first-frame, steady frame, CPU/GPU; transport passes on eval host, broader app metrics still missing.
P1	API coverage	real/stateful/safe-stub table; partial.
P1	visual artifacts	screenshots or frame hashes; native CRC only.
P2	robustness tests	malformed resource/fence/capset cases; future.

introduce command-encoding, serialization, host-visible memory, and renderer-latency overheads. The right future evaluation is therefore not “VOGUE faster than Linux” in the abstract, but a breakdown of boot, memory, frame latency, and CPU utilization across: VOGUE software render, VOGUE virgl, Linux DRM/Mesa, and a minimal Linux appliance.

9.6 Threats to Validity

External validity. The kmscube, scene-clear, and null-renderer workloads are intentionally small. They show feasibility, not coverage of complex GUI or game workloads.

Measurement validity. Current footprint and boot comparisons are useful, but a submission-quality evaluation should pin CPU frequency, repeat runs, report variance, and publish raw logs.

Implementation validity. The fake backend is excellent for command-order tests and the static gate is useful for ABI coverage, but neither substitutes for QEMU/virgl/Venus behavior. All claims that depend on host-device behavior must be backed by same-run QEMU logs and frame evidence.

9.7 Remaining Work for SOSP-Level Competitiveness

Table 19 summarizes the reviewer-driven work that cannot honestly be claimed by the current artifact. VOGUE now fixes the paper-structure, figure, and native application-performance weaknesses, but the table keeps the larger implementation gaps visible.

9.8 Root Cause Analysis of the Remaining Gaps

9.8.1 K1: current PASS and remaining scope

K1 is a current PASS claim on the evaluation host. The root causes that previously blocked it, and the guardrails that remain, are:

1. **Native encoder readiness is separate from QEMU runtime proof.** `virgl_encoder.c` in `libkvmvirtio_gpu` encodes `CREATE_OBJECT/SURFACE`, `SET_FRAMEBUFFER_STATE`, and `CLEAR` command streams, and the native `virgl_encoder_test` passes. This proves command construction under the fake backend, not host `virglrenderer` acceptance.
2. **Current run log must carry the required PASS marker.** `kmscube_vgpu_gl_eval.py` removes stale `frame-proof.json` data when it is not present in the current `run.log`. Submit PASS requires the current K1 row to carry `renderer=virgl` and `submits_3d>0`.

3. Pixel proof is stricter than submit proof.

`frame_pixel_proof.json` may record `SUBMIT_3D` diagnostics, but `gfx.kmscube.frame` only passes when the screen-dump itself matches an encoded CLEAR colour band and the proof hashes the same `run.log`.

The reproduction plan is to rebuild the KMSCube appliance with KraftKit, run it under QEMU’s `virtio-gpu-gl/Venus`-capable backend as documented by QEMU [25], capture a same-run serial PASS marker, capture the QMP screen-dump, and rerun `make kmscube-check && make eval-check`. Without that run on a new host, native `virgl` encoder results remain substrate evidence only.

9.8.2 llama.cpp full inference: current PASS and remaining scope

Full upstream `llama.cpp` CPU and Vulkan inference now pass inside single-purpose Unikraft appliances on the evaluation host. The project has already crossed the major bring-up milestones: C++ runtime support, POSIX threads, model delivery via 9pfs, static Vulkan dispatch for `ggml-vulkan`, and real QEMU/Venus runtime execution. The remaining questions are therefore not “can the guest run upstream `llama.cpp`?” but rather:

1. **Can the server path serve real requests?** Today the server images prove direct endpoint plus model-loaded readiness, not request/response service.
2. **Can model load be reduced further?** Current artifacts still record `use_mmap=false` and `huge_pages=false`.
3. **How much of the remaining overhead belongs to the guest path versus the transport and host stack?** The repaired Vulkan throughput is strong, but a fuller server-level comparison still requires request gates and matched baselines.

9.9 Future Work

The current artifact has 27 generated evidence rows: 27 PASS and 0 blocked on the evaluation host. The N1/N2/`vk.ggml-dispatch` milestones described above are now complete for runtime bring-up:

Completed (this stage):

- N1 CPU path: `apps/app-llama-upstream` with upstream-unmodified `llama.cpp` running inside Unikraft. `llm.bench.cpu` PASS: `pp512=9.1 t/s`, `tg128=7.7 t/s`.
- N2 model delivery: 9pfs host-share via `VirtIO-9P` (`CONFIG_LIBUK9P=y`, `CONFIG_LIB9PFS=y`,

CONFIG_LIBVFSSCORE_AUTOMOUNT_CI_RAMFS=y). Model mounted at /mnt/model.

- vk.ggml-dispatch static dispatch and runtime: apps/app-llama-upstream-vk with libukggml_vulkan builds, dispatch tests pass, and the Vulkan bench/server appliances have same-run Venus artifacts.
- POSIX surface served entirely by upstream lib-musl external library on top of Unikraft’s posix-mmap/time/fdio/vfs libs — no project-local POSIX shim is required.

Next stage:

1. **HTTP server evidence.** llm.server.vk now reaches READY with batch_size=2048, ubatch_size=512, and prompt caching enabled, but there is still no lwIP/netdev-backed request path. Add request serving before reporting TTFT, requests/s, or aggregate server throughput.
2. **Model-load optimization.** model-load-time-check still records use_mmap=false, huge_pages=false; the remaining 4.7-7.1 s model-load path should be reduced before claiming startup wins for larger models.
3. **Linux/QEMU baseline.** Collect matched kmscube/glmak2 runs on Linux+Mesa under the same host for a direct performance comparison.
4. **Full glmak2 scene coverage and frame hashes.** Replace the current scene-clear substrate proof with at least one real visual workload, frame hashes, and a Linux Mesa baseline.
5. **Security audit.** Quantify attack surface reduction, fuzz the command parser, and evaluate malicious-device behavior.

10 Related Work

10.1 Unikernels and Library OSes

VOGUE builds directly on the unikernel and library-OS tradition. MirageOS argued that single-purpose cloud services can be built as type-safe library OSes with small images and strong specialization [16]. Unikraft later generalized the idea with a micro-library architecture, Kconfig composition, and evidence that specialization can improve footprint and performance for server workloads [12]. Drawbridge explored library OS compatibility from the opposite direction: preserve rich Windows application compatibility by moving a large OS personality into an isolated library OS [22]. VOGUE takes the Unikraft/Mirage side of this tradeoff: it does not import a rich OS personality, but instead implements the smallest graphics personality needed by selected workloads.

The Exokernel line of work exposed low-level resources to applications for application-specific management [5]. Arakis and IX similarly split control-plane policy from data-plane I/O, allowing applications to bypass kernel mediation on the fast path [2, 21]. VOGUE is narrower: it does not redesign the whole OS I/O model, but it applies the same principle to graphics in a unikernel. The VirtIO-GPU data

path becomes a library-level fast path, while the hypervisor and device protocol remain the protection boundary.

10.2 Lightweight Virtualization

Firecracker demonstrates that microVMs can provide fast startup and strong isolation for serverless workloads [1]. VOGUE is complementary: it targets what runs inside a minimal VM when the appliance needs graphics. The key question is not whether a VM can be lightweight, but whether the guest graphics stack can be made lightweight enough to preserve unikernel advantages.

10.3 GPU Virtualization

VirtIO-GPU is the device target. The protocol is specified by OASIS [20] and implemented by QEMU [25]. VirGL translates guest Gallium-style command streams to host OpenGL [30]. Venus provides a Vulkan-over-VirtIO path, and rutabaga/gfxstream supports ChromeOS/Android graphics virtualization [3, 18]. The vhost-user-gpu mechanism externalizes the GPU backend for process isolation [24].

These systems primarily define host/device mechanisms or full guest-stack integration. VOGUE’s contribution is guest-side: a small Unikraft library stack that can speak the device protocol and host selected graphics applications without Linux DRM or Mesa. The current artifact advances beyond transport-only bring-up by showing same-run guest Vulkan execution for upstream *llama.cpp*, while still leaving broader graphics compatibility and richer scene coverage outside its scope.

10.4 Minimal Graphics Stacks

Linux DRM/KMS [14], Mesa [17], GBM, EGL, and GLES form the commodity path for direct-rendered applications. VOGUE replaces this stack with a compatibility boundary, not a complete reimplement. Headless software renderers such as llvmpipe or OSMesa provide software pixels through Mesa; VOGUE’s rasterizer is intentionally smaller and less general. The tradeoff is explicit: VOGUE gains auditability and small footprint, but loses shader and compiler coverage until virgl is implemented.

10.5 Application Porting to Unikernels

Prior Unikraft application work focuses on network and storage services. VOGUE extends the porting pattern to graphics by factoring the port into reusable shims rather than application-specific rewrites. The closest analogy is POSIX compatibility work: a narrow API surface enables unmodified source to compile, but every stub must be accounted for. VOGUE makes that accounting visible through claim rows and forbidden claims.

11 Conclusion

This paper demonstrates that a unikernel can support practical graphics display workloads without importing a full Linux graphics subsystem. VOGUE implements a standards-based VirtIO-GPU 2D pipeline, a minimal EGL/GLES2/GBM/DRM shim, and a CPU software rasterizer as Unikraft micro-libraries. It also implements the real VirtIO-GPU

3D/blob control-queue surface needed for the next accelerated stage while preserving Unikraft’s design principle of reusing existing PCI, virtio, allocator, and POSIX/mmap support instead of reimplementing them.

Our evaluation validates progressive milestones: (disp.2d) a correct 2D display pipeline verified by native evidence-gated tests; (gfx.kmscube.sw) upstream *kmscube* source compiled and running against the shim with software rendering; (gfx.glmark2.sw) an additional application framework demonstrating substrate generality; (proto.real-driver, vk.readiness) real-driver ABI/readiness and benchmark-design gates; (vk.drm-shim, gfx.vkmark, vk.ggml-dispatch) Venus/Vulkan and ggml substrate gates; and, on the evaluation host, real QEMU/Venus transport, KMSCube SUBMIT_3D/pixel proof, and upstream llama.cpp Vulkan runtime with post-fix same-run `pp512=2232.1` and `tg128=160.2` over Venus. The remaining gaps are HTTP serving, broader vkmark scene rendering, the llvmpipe-specific ENV9 artifact, and model-load optimization rather than decode-path rescue.

The remaining work is no longer first transport proof; it is breadth, comparison, and performance characterization. The infrastructure for the guest graphics/Vulkan substrate — context management, blobs, map/unmap, Venus encoding, ring protocol, static Vulkan dispatch, and bounded application shims — is now in place and backed by current evidence. The next concrete step is to expand from bounded demonstrations to broader accelerated workloads, matched Linux baselines, and request-level LLM serving experiments, while preserving the same evidence discipline that kept weaker artifacts from being mistaken for stronger claims. That discipline is itself part of the paper’s contribution: it offers a practical methodology for extending unikernels into graphics- and GPU-adjacent domains without overclaiming along the way.

References

- [1] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: Lightweight Virtualization for Serverless Applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020. 419–434.
- [2] Adam Belay, George Prekas, Ana Klimovic, Samuel Grossman, Christos Kozyrakis, and Edouard Bugnion. 2014. IX: A Protected Dataplane Operating System for High Throughput and Low Latency. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2014. 49–65.
- [3] ChromiumOS contributors. 2024. Rutabaga: gfxstream graphics abstraction layer. Retrieved from https://crosvm.dev/doc/rutabaga_gfx/index.html
- [4] Rob Clark and Arvin Schnell. 2023. kmscube: DRM/KMS cube demo. Retrieved from <https://gitlab.freedesktop.org/mesa/kmscube>
- [5] Dawson R. Engler, M. Frans Kaashoek, and James O’Toole. 1995. Exokernel: An Operating System Architecture for Application-Level Resource Management. In *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles (SOSP)*, 1995. 251–266. <https://doi.org/10.1145/224056.224076>
- [6] ggml-org. 2024. llama.cpp: LLM Inference in C/C++.
- [7] ggml-org. 2024. GGML VirtGPU Backend.
- [8] glmark2 contributors. 2024. glmark2: OpenGL (ES) 2.0 benchmark suite. Retrieved from <https://github.com/glmark2/glmark2>
- [9] The Khronos Group. 2024. OpenGL: The Industry’s Foundation for High Performance Graphics.
- [10] The Khronos Group. 2024. Vulkan: Cross-Platform 3D Graphics and Compute.
- [11] The Khronos Group. 2026. Vulkan Guide: Loader. Retrieved from <https://docs.vulkan.org/guide/latest/loader.html>
- [12] Simon Kuenzer, Vlad-Andrei Bădoiu, Hugo Lefeuvre, Sharan Santhanam, Alexander Jung, Gauthier Gain, Cyril Soldani, Costin Lupu, Ștefan Teodorescu, Costi Răducanu, Cristian Banu, Laurent Mathy, Răzvan Deaconescu, Costin Raiciu, and Felipe Huici. 2021. Unikraft: Fast, Specialized Unikernels the Easy Way. In *Proceedings of the 16th European Conference on Computer Systems (EuroSys)*, 2021. <https://doi.org/10.1145/3447786.3456248>
- [13] Joost Lim and others. 2022. Cricket: CUDA Virtualization via API Remoting.
- [14] Linux kernel contributors. 2024. Direct Rendering Manager (DRM/KMS) subsystem. Retrieved from <https://www.kernel.org/doc/html/latest/gpu/>
- [15] Linux Kernel Documentation. 2024. VFIO – Virtual Function I/O.
- [16] Anil Madhavapeddy, Richard Mortier, Charalampos Rotsos, David Scott, Balraj Singh, Thomas Gazagnaire, Steven Smith, Steven Hand, and Jon Crowcroft. 2013. Unikernels: Library Operating Systems for the Cloud. In *Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2013. 461–472. <https://doi.org/10.1145/2451116.2451167>
- [17] Mesa contributors. 2024. Mesa 3D graphics library. Retrieved from <https://mesa3d.org/>

- [18] Mesa contributors. 2024. Venus: Vulkan over VirtIO. Retrieved from <https://docs.mesa3d.org/drivers/venus.html>
- [19] NVIDIA Developer. 2023. Confidential Computing on NVIDIA H100 GPUs for Secure and Trustworthy AI.
- [20] OASIS Technical Committee. 2023. *Virtual I/O Device (VIRTIO) Version 1.3*. Retrieved from <https://docs.oasis-open.org/virtio/virtio/v1.3/virtio-v1.3.html>
- [21] Simon Peter, Jialin Li, Irene Zhang, Dan R. K. Ports, Doug Woos, Arvind Krishnamurthy, Thomas Anderson, and Timothy Roscoe. 2014. Arrakis: The Operating System is the Control Plane. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2014. 1–16.
- [22] Donald E. Porter, Silas Boyd-Wickizer, Jon Howell, Reuben Olinsky, and Galen C. Hunt. 2011. Rethinking the Library OS from the Top Down. In *Proceedings of the 16th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2011. 291–304. <https://doi.org/10.1145/1950365.1950399>
- [23] QEMU Documentation. 2024. AMD Secure Encrypted Virtualization (SEV).
- [24] QEMU Project. 2024. vhost-user-gpu: External VirtIO-GPU backend. Retrieved from <https://www.qemu.org/docs/master/interop/vhost-user-gpu.html>
- [25] QEMU Project. 2026. virtio-gpu — QEMU Documentation. Retrieved from <https://www.qemu.org/docs/master/system/devices/virtio-gpu.html>
- [26] Typst Project. 2026. Typst PDF Export Documentation. Retrieved from <https://typst.app/docs/reference/pdf/>
- [27] Unikraft Project. 2026. kraft build — Configure and build Unikraft unikernels. Retrieved from <https://unikraft.org/docs/cli/reference/kraft/build>
- [28] Unikraft Project. 2026. Kraftfile Reference. Retrieved from <https://unikraft.org/docs/cli/reference/kraftfile/latest>
- [29] vAccel Project. 2024. vAccel: A Hardware-Agnostic Acceleration API.
- [30] virglrenderer contributors. 2024. virglrenderer: A virtual 3D GPU library. Retrieved from <https://gitlab.freedesktop.org/virgl/virglrenderer>

Table 20: Current artifact claim map. The evaluation host records 27 PASS rows, but new hosts must still generate the corresponding same-run artifacts before promoting transport or runtime claims.

Paper claim / row family	Artifact component	Validation source
2D display pipeline	native 2D render test	make native-tests PASS log
VirtIO-GPU wire ABI	make -C tests proto-abi	struct/offset/feature checks against virtio_gpu_proto.h
Software graphics rows	application perf scripts + app-kmscube / app-g1mark2	make app-perf-check
VirtIO-GPU real-driver readiness	real backend static/readiness checks	make venus-check
Venus wire-format + ring protocol	venus_cs_test, registry checks, ring-protocol tests	make native-tests, make venus-check
vk.drm-shim / vk.icd substrate	virtgpu_drm_ioctl_test, vk_icd_bootstrap_test	make native-tests
Static ggml-Vulkan dispatch	ggml_vk_dispatch_test, API coverage scripts	make llama-vulkan-api-coverage, make llama-ggml-vk-dispatch
QEMU/Venus transport	QEMU probe + runtime artifacts	make venus-check, evaluation matrix rows
Upstream llama.cpp CPU appliance	results/llama/upstream_cpu.json	make llama-upstream-cpu-check
Upstream llama.cpp Vulkan appliance	results/llama/upstream_vk.json, post_opt_runs/	make llama-upstream-vk-check + same-run runtime logs
Upstream llama.cpp Vulkan server readiness	results/llama/upstream_server_vk.json	make llm-server-vk-check
Current-stage consistency	results/stage/current_stage_report.json	make current-stage-check

12 Artifact Appendix

12.1 Artifact Overview

The VOGUE artifact contains the Unikraft VirtIO-GPU library stack, application ports, build scripts, native test suite, evaluation scripts, paper-generation scripts, and raw result logs used to produce the evidence matrix in this paper. The artifact is deliberately evidence-gated: software-render display results, real-driver ABI/readiness checks, QEMU/Venus transport status, native substrate performance, and future accelerated-rendering claims are separate rows.

12.2 Artifact Claims

Generated performance and stage artifacts are written to results/ and pulled into the paper through generated tables. The central rule is unchanged across all tiers: if the host, build, or runtime environment cannot support a claim, the artifact records a structured blocker instead of promoting a weaker result.

12.3 External Dependency Inventory

All external code referenced by this artifact is listed below with exact local paths and versions. This table allows a reviewer to verify reproducibility and identify what must be installed before running any gate.

12.3.1 Venus Protocol Header Inventory

The Venus protocol wire format is defined by Mesa’s vn_protocol_driver_defines.h. The following tokens from that header are the load-bearing pieces for VOGUE’s encoder correctness:

12.4 Hardware and Software Requirements

VOGUE’s evidence rows span three tiers with different hardware requirements. The native-test and software-render tiers require only a Linux host; the Venus/Vulkan

tier additionally requires a Vulkan-capable host GPU and QEMU 11.0.

12.5 Official-Documentation-Grounded Environment Plan

The build environment follows the upstream documentation rather than project-local assumptions. KraftKit’s official kraft build reference defines target selection by platform/architecture (for example qemu/x86_64) [27], and the Kraftfile reference defines targets as platform/architecture tuples [28]. QEMU documents the relevant graphics backend as virtio-gpu-gl with hostmem, blob=true, and venus=true for Venus/Vulkan [25]; Mesa documents Venus as Vulkan command serialization over VirtIO-GPU [18]. Khronos documents that the Vulkan loader maps applications to layers and installable client drivers [11]. Typst documents that compile produces the paper PDF [26].

12.6 Build and Evaluation Instructions

The full local verification gate runs native tests, application substrate benchmarks, paper compilation, claim audit, and QEMU/Kraft blocker capture.

The native test suite is organized into five focused groups. Reviewers can reproduce individual evidence chains without running the full suite:

Expected developer test output includes all host-native substrate tests plus the ABI guard:

Expected application substrate output:

If the external Kraft/Unikraft appliance build or QEMU/Venus probe fails, the target records a structured blocker rather than promoting K1. A K1 pass requires a same-run log with an accepted VirtIO-GPU device, capset/feature evidence, 3D or Venus command submission, frames ≥ 3 , and frame hashes or screenshots tied to the run.

Table 21: External dependency inventory. Paths are referenced through the variables defined in `config/external_paths.json`; the defaults match the evaluation host and can be overridden via the corresponding environment variables. The `cmake/unikraft-clang.cmake` toolchain file consumes the same variables.

Component	Local Path	Version / SHA	Purpose
<i>Mesa Venus protocol headers</i>	<code>\$VENUS_PROTOCOL_ROOT/include/</code>	KhronosGroup/venus-protocol	Venus wire-format type definitions; packed struct layout
<i>Vulkan Headers</i>	<code>\$VULKAN_HEADERS_INCLUDE</code>	Khronos Vulkan-Headers v1.3.352	Vulkan API headers for <code>libukgml_vulkan</code> , <code>libukvk_icd</code> , <code>apps/app-llama-upstream-vk</code>
<i>SPIR-V Headers</i>	<code>\$SPIRV_HEADERS_INCLUDE</code>	KhronosGroup/SPIRV-Headers (matched to Vulkan 352)	SPIR-V encoding enums for <code>gml-vulkan.cpp</code> shader table
<i>llama.cpp (upstream)</i>	<code>\$LLAMA_ROOT</code> (host build); in-tree at <code>apps/app-llama-upstream*/</code>	SHA <code>fcae601e4</code> (ggml-org/llama.cpp)	CPU baseline benchmarks; upstream-unmodified Unikraft port; ggml-vulkan compute
<i>libvulkan.so</i>	<code>/usr/lib/x86_64-linux-gnu/libvulkan.so.1</code>	system Vulkan loader	Host-side Vulkan tests (make <code>vulkan-tests</code>)
<i>virglrenderer</i>	host QEMU dependency	linked into QEMU 11.0	Host-side Venus capset decode and <code>SUBMIT_3D</code> dispatch
<i>QEMU</i>	<code>qemu-system-x86_64</code>	11.0 (required for Venus capset id=4)	VM execution for all appliance eval targets
<i>KraftKit / kraft</i>	<code>kraft</code> in <code>PATH</code>	v0.9+ recommended	Unikraft appliance build: <code>make kmscube-build</code> , <code>make llama-vulkan-n3-build</code>
<i>Unikraft tree</i>	<code>../unikraft</code> (sibling checkout)	aligned with Kraftfiles in <code>kraft/</code>	Unikraft kernel and library build system
<i>CMake toolchain</i>	<code>cmake/unikraft-clang.cmake</code>	in-tree	Forces SPIR-V + Vulkan v352 headers for clang cross-builds
<i>typst</i>	<code>typst</code> in <code>PATH</code>	v0.11+	Paper compilation: <code>make paper</code>

Table 22: Venus protocol header tokens verified in the VOGUE artifact. The `real_virtio_gpu_path_check.py --check` gate fails if any token is absent from `libs/libukvirtio_gpu/virtio_gpu_proto.h`.

Token / Field	Significance for VOGUE
<code>VN_CS_ENCODER_BUFFER_SIZE</code>	Ring command-buffer segment size; must match <code>virglrenderer</code> decoder expectations
<code>vn_encode_array_size(enc, n)</code>	Encodes array presence as <code>uint32_t n</code> ; zero means absent; VOGUE encoder replicates this protocol in <code>libukvenus</code>
<code>VN_ENCODE_* PACKED layout</code>	No inter-field alignment padding; all structs use <code>__attribute__((packed))</code> or equivalent; <code>virglrenderer</code> assumes this
<code>VIRTIO_GPU_F_VIRGL</code> , <code>VIRTIO_GPU_F_RESOURCE_BLOB</code>	Feature bits negotiated at probe; required before any 3D or blob command is issued
<code>VIRTIO_GPU_CMD_SUBMIT_3D</code> , <code>VIRTIO_GPU_CMD_RESOURCE_CREATE_BLOB</code> , <code>VIRTIO_GPU_RESP_OK_MAP_INFO</code>	Control-queue command IDs; verified by <code>real_virtio_gpu_path_check.py</code>

12.7 Troubleshooting

12.8 Artifact Badge Target

We target **Artifacts Available** and **Artifacts Evaluated – Functional** for the bounded PASS claims in the current generated matrix. The current artifact has 27 evidence rows: 27 PASS and 0 blocked on the evaluation host, including `xport.qemu-vgpu`, `gfx.kmscube.submit/gfx.kmscube.frame`, `host.vk.probe`, `host.bench.vk.run`, `host.bench.vk`, `llm.bench.vk`, `llm.server.vk`, and `llm.bench.vk.real`. The artifact still requires fresh row-com-

patible same-run artifacts before promoting those rows on any different host.

12.8.1 Reproduction Notes for the Vulkan Path

The pinned upstream `ggml-vulkan.cpp` API surface is covered by the static dispatch table and checked by `make llama-vulkan-api-coverage`. Reproducing the runtime path additionally requires a Venus-capable QEMU stack plus a readable render node. On the evaluation host, those requirements are satisfied and the same-run artifacts are already present; on a

Table 23: Hardware and software requirements by tier. Source for Venus host GPU requirements: Mesa Venus documentation at <https://docs.mesa3d.org/drivers/venus.html>. Any Vulkan 1.1 GPU driver on Linux is sufficient; no specific GPU vendor or model is required.

Component	Minimum Requirement	Notes
Host CPU architecture	x86_64 (primary); arm64 untested	KVM optional for native-test tier; recommended for appliance tier
Host OS	Linux (kernel ≥ 5.4 for VirtIO-GPU 2D; kernel ≥ 5.16 for blob/Venus features)	Ubuntu 22.04 or later recommended; any distro with QEMU 11.0 available
Host GPU — native and sw-render tiers	None required	<code>make native-tests</code> , <code>make app-perf-check</code> , <code>make eval-check</code> , <code>make paper run</code> without any GPU
Host GPU — Venus/Vulkan tier	Any GPU with a Vulkan 1.1 driver on Linux	Tested drivers per Mesa Venus documentation: ANV ≥ 21.1 , RADV ≥ 21.1 , NVIDIA proprietary ≥ 570.86 , Turnip ≥ 22.0 , Lavapipe ≥ 22.1 (software). Requires <code>VK_KHR_external_memory_fd</code> . GPU vendor, architecture, and model are not constrained beyond Vulkan 1.1 support.
VMM	QEMU ≥ 7.0 for 2D VirtIO-GPU; QEMU 11.0 for Venus capset id=4	Must be built with <code>virglrenderer</code> support (<code>--enable-virglrenderer</code>). Venus requires <code>virtio-gpu-gl-pci,blob=true,venus=true</code> . The recorded probe used QEMU 11.0.
QEMU KVM flag	<code>-accel kvm,honor-guest-pat=on</code> recommended for Venus	Per Mesa Venus documentation: <code>honor-guest-pat=on</code> required for correct Venus host-visible memory behavior. KVM without this flag may cause coherency issues.
virglrenderer	Version linked into QEMU build	Must include Venus support (Vulkan/capset-4 path). No separate minimum version number specified; use the version distributed with your QEMU 11.0 build.
Guest OS	Unikraft (this repository)	Unikraft replaces Linux DRM/KMS; guest kernel ≥ 5.16 requirement applies to Linux guests only
Host Vulkan loader	<code>libvulkan.so.1</code> for host-side Vulkan tests	Used by <code>make vulkan-tests</code> and <code>tests/vk_icd_bootstrap_test</code> ; not required for native-test tier
Build toolchain	clang or gcc; KraftKit + sibling <code>../unikraft tree</code>	clang/gcc for native tests; KraftKit for appliance builds; <code>typst</code> for paper

Table 24: Environment, problem, root-cause, and fix plan grounded in upstream documentation and current generated artifacts.

Task / problem	Observed reason	Fix / verification plan
Kraft/QEMU appliance build	Requires a KraftKit target and sibling Unikraft tree; interrupted or missing builds leave runtime rows blocked.	<code>kraft build --target qemu/x86_64</code> through <code>make kmscube-build</code> ; then <code>make kmscube-check</code> .
QEMU/Venus runtime	QEMU must provide a GL/Venus-capable VirtIO-GPU backend with <code>hostmem/blob/Venus</code> enabled; otherwise QEMU probes/runtime rows stay blocked:*	Install/choose QEMU with <code>virglrenderer/Venus</code> , use a GL-capable display backend, then rerun <code>make venus-check</code> and <code>make eval-check</code> .
KMSCube submit/frame proof	Native encoder tests are not same-run host proof; stale frame artifacts must not pass.	Require current run-log PASS marker plus pixel colour-band proof; run <code>make kmscube-check</code> && <code>make eval-check</code> .
Vulkan/LLM runtime rows	Host baseline JSON is the wrong domain for Unikraft runtime claims.	Require row-compatible <code>evidence_id</code> , Unikraft run source, and required fields before PASS; same-run Unikraft artifacts now exist on the evaluation host.
Paper build	Typst and fonts are host tools; missing fonts are warnings if <code>typst compile</code> exits zero.	<code>make paper-check paper</code> ; treat non-zero Typst exit as failure.
Performance noise	Native software/substrate timing varies on non-isolated hosts.	<code>make perf-check</code> ; best-of-N is the smoke-gate value, JSON persists median/all samples for analysis.

different host, the same commands either regenerate PASS artifacts or emit structured blockers.

```
# Clone with submodules
git clone <repo> vogue && cd vogue
```

Listing 2: Build and evaluation commands. Run “make verify” for the complete gate; use individual targets to reproduce specific evidence rows.

```
# Full local verification gate
make verify

# For production (e.g. CI) or CI (e.g. CI)
make test-fast
```

```
# Individual gates used by reviewers
make native-tests
```


Table 25: Native test group targets. All run on the host without QEMU, Unikraft, or a GPU. See `tests/README.md` for expected pass strings and evidence mapping.

Target	Test group	Evidence rows
<code>make test-core</code>	VirtIO-GPU core, DMA, shims	<code>proto.real-driver</code> , <code>gfx.kmscube.sw</code> , <code>disp.2d</code>
<code>make test-venus</code>	VirtIO-GPU 3D, Venus encoder, virgl	<code>vk.drm-shim</code> , <code>vk.icd</code> , <code>proto.venus-enc</code> , <code>proto.venus-ring</code>
<code>make test-llm</code>	LLM/ggml substrate	<code>llm.bench.cpu</code> , <code>llm.bench.vk</code> , <code>vk.ggml-dispatch</code>
<code>make test-dispatch</code>	<code>vk.ggml-dispatch</code> static Vulkan dispatch (164 checks)	<code>vk.ggml-dispatch</code>
<code>make proto-abi</code>	VirtIO wire-ABI struct/offset/feature	<code>proto.real-driver</code>

```
make native-tests
dma_buf_test passed alignment=4096 sg=1 pool=2
virtio_gpu_full_api_test passed capsets=5 fences=5
submits_3d=1 blobs=1
virtio_gpu_2d_render_test: PASS frames=3 transfers=3
flushes=3 fences=6
kmscube_compat_test passed mode=1280x800
virtgpu_drm_ioctl_test: all checks passed
vk_icd_bootstrap_test: all checks passed
venus_cs_test: all checks passed
venus_compute_test: all checks PASS
virgl_encoder_test: all checks passed
ukmodel_test: all checks passed
ggml_uk_test: all checks passed
ukllama_test: all checks passed
ggml_vk_dispatch_test: all checks PASS [164 passed, 0
failed]
make -C tests proto-abi
virtio_gpu_proto_abi_test passed ctrl_hdr=24
display_info=408 edid=1056
```

Listing 3: Expected `make native-tests` output. All listed native tests plus the ABI guard pass without QEMU, a GPU, or prebuilt ggml libraries.

```
gfx.kmscube.sw: kmscube avg_frame_ms=<measured>
fps=<measured> transfers=60 flushes=60 fences=120
gfx.glmark2.sw: glmark2 scene clear
avg_frame_ms=<measured> ...
```

Listing 4: Expected application substrate benchmark output. Measured values vary by host; the structure and row labels must match.

Table 26: Common issues and fixes.

Symptom	Likely Cause	Fix
<i>native tests fail</i>	missing compiler/header path	check tests/Makefile and production library includes
<i>kmscube-build records blocked build</i>	external Kraft/Unikraft compiler mismatch	inspect the build log; fix toolchain before claiming K1
<i>black QEMU display</i>	scanout or flush not issued	verify SET_SCANOUT, TRANSFER_TO_HOST_2D, and RESOURCE_FLUSH order
<i>virtio-gpu-gl init fails</i>	no host OpenGL/virglrenderer, or virglrenderer not linked into QEMU	use 2D software path or install GL-capable QEMU stack built with --enable-virglrenderer
<i>venus probe regresses to blocked:modern-pci-unsupported</i>	Unikraft libvirtio_pci patch for modern VirtIO-GPU ID 0x1050 missing or not selected	apply/reuse the modern virtio-pci support patch before claiming xport.qemu-vgpu; still require K1 frame proof before claiming Venus acceleration
<i>blocked:egl-not-initialized or blocked:no-pass-line on Venus/Vulkan targets</i>	missing image, missing KVM/Venus support, or stale evidence from old QEMU EGL launch	Use a GL-capable QEMU display backend (for example egl-headless, gl=on) on a host with a working render node; inspect structured JSON before promoting any claim. These blockers are resolved on the evaluation host.
<i>Host Vulkan 1.1 not available</i>	Host GPU driver older than listed minimums, or Vulkan loader not installed	Install a Vulkan 1.1 driver (any of: ANV ≥ 21.1, RADV ≥ 21.1, NVIDIA proprietary ≥ 570.86, Lavapipe ≥ 22.1). vulkaninfo confirms capabilities. VK_KHR_external_memory_fd must be present.

```

make llama-vulkan-api-coverage # required API surface
make llama-ggml-vk-dispatch   # native static-dispatch
substrate
make llama-upstream-vk-build   # build the Vulkan
appliance
make llama-upstream-vk-run      # runtime PASS or
structured blocker
make llm-server-vk-check        # Vulkan server readiness
contract

```

Listing 5: Verification commands for the ggml-Vulkan/Venus path. Runtime claims require same-run PASS evidence; blocked rows remain non-claims.

13 GPU Driver Ecosystem Survey

This appendix surveys the landscape of GPU virtualization protocols, host-side rendering backends, guest graphics stacks, and emerging compute-remoting approaches, placing VOGUE’s current implementation in context at each layer. The goal is to map what exists, characterize where VOGUE intersects each technology, and identify the dependency chain required to advance blocked evidence rows.

13.1 Virtualization Substrate

VirtIO and VirtIO-GPU. VirtIO [20] establishes the standard para-virtualization device model: feature negotiation at probe time, shared virtqueue descriptor rings, and asynchronous notifications. VirtIO-GPU extends this model with a dedicated GPU device class that multiplexes a control queue and a cursor queue over the same transport. VOGUE’s `libukvirtio_gpu` implements the guest side of this protocol directly against Unikraft’s VirtIO transport, avoiding any Linux kernel infrastructure. The current implementation covers the full 2D command surface: `RESOURCE_CREATE_2D`, `RESOURCE_ATTACH_BACKING`, `TRANSFER_TO_HOST_2D`, `SET_SCANOUT`, `RESOURCE_FLUSH`, and fence synchronization. This is sufficient for evidence rows `disp.2d` and `gfx.kmscube.sw`.

VirtIO-GPU 3D / `VIRTIO_GPU_F_VIRGL`. The 3D feature bit promotes the device from a framebuffer adapter to an opaque command-stream forwarder. When negotiated, the guest may issue `CTX_CREATE`, `GET_CAPSET_INFO`, `GET_CAPSET`, and `SUBMIT_3D` commands; the host forwards the command buffer to `virglrenderer` for OpenGL translation. VOGUE’s `libukvirtio_gpu` implements the transport commands for context lifecycle, resource attach/detach, and 3D submission, but the guest still lacks the final accelerated-frame path: `virgl` command-stream coverage for OpenGL and Venus frame-proof wiring for Vulkan. This is the K1 blocked row. The former QEMU 11.0 modern VirtIO-PCI discovery blocker for device ID `0x1050` is resolved by the Unikraft patch carried in this artifact.

Resource blobs and host-visible memory. VirtIO-GPU 1.1 introduced resource blobs: a mechanism for creating resources backed by host memory, guest memory, or shared guest-host memory without an explicit `TRANSFER_TO_HOST_2D` round-trip. Blob resources are the transport foundation for Venus (Vulkan) and for high-bandwidth ML-weight loading. VOGUE now implements the blob, UUID, map, and unmap control-queue commands in the real backend and validates their ABI/static presence. This is necessary Venus-readiness evidence, but it is not a Vulkan application run until the real appliance command stream is exercised in QEMU and produces frame/host-processing evidence.

13.2 Host-Side Rendering and Backend Isolation

QEMU `virtio-gpu` and `virtio-gpu-gl`. QEMU [25] exposes VirtIO-GPU through multiple device variants. The baseline

`virtio-gpu` device handles 2D commands and provides a software display without host GPU involvement. The `virtio-gpu-gl/Venus-capable` variants link against `virglrenderer` and route 3D or Vulkan command streams to host OpenGL/Vulkan drivers. VOGUE targets these variants in stages: the 2D software path runs today; the real 3D/blob controlq surface is implemented; the current generated matrix records QEMU/Venus transport as PASS on the evaluation host; the native and QEMU Venus ring-buffer protocol passes; and the remaining work is broader rendering coverage and performance.

VirGL and `virglrenderer`. VirGL [30] is a translation layer that interprets guest Gallium-style command streams and emits corresponding host OpenGL calls. It operates entirely on the host; the guest is responsible for generating syntactically and semantically correct commands. This division of responsibility is important for VOGUE: the guest command encoder and the host renderer are independent components. A bug in the guest encoder produces `virglrenderer` errors or silently wrong frames; it cannot be debugged by examining `virglrenderer` source alone.

Venus. Venus [18] provides a VirtIO-GPU Vulkan serialization protocol. Guest Vulkan API calls are serialized by the Mesa guest driver and forwarded through VirtIO-GPU command streams; `virglrenderer` deserializes and dispatches them as host Vulkan calls. Venus requires blob resources and host-visible memory for zero-copy command and buffer sharing. VOGUE now targets the Venus transport substrate — blob resources, context operations, UUIDs, map/unmap, and `SUBMIT_3D` — with verified wire-format correctness: the encoder uses PACKED layout (no inter-field alignment padding) and encodes array presence as `uint32 array_size` values per Mesa’s `vn_encode_array_size()` protocol, matching `virglrenderer`’s decoder (`vk_context_submit_cmd`). The current evaluation host proves the QEMU/Venus path with same-run transport, frame, and `llama.cpp` Vulkan runtime artifacts; broader Mesa/Venus guest-driver coverage remains future work.

Rutabaga and `gfxstream`. Rutabaga [3] is a Rust graphics virtualization abstraction layer developed for ChromeOS and Android. It unifies 2D, VirGL, Venus, and `gfxstream` paths behind a single host API, enabling `crosvm` and QEMU to share backend implementations. `Gfxstream` is a distinct protocol that serializes GLES and Vulkan calls at the API level rather than at the Gallium or `VkCommand` level, targeting low-latency Android use cases. VOGUE has no direct dependency on `rutabaga`, but `rutabaga` is the authoritative reference for how blob, `hostmem`, and `capset` semantics should behave when `virgl` is implemented.

vhost-user-gpu. The `vhost-user-gpu` mechanism [24] moves the graphics backend out of QEMU into a separate process, communicating over a UNIX socket with file-descriptor passing for `DMABUF` sharing. This isolation boundary allows the backend renderer to crash or be replaced

without affecting the VMM or guest. It is the preferred architectural boundary for production deployments and is the natural host-side target for a GGML-VirtGPU-style compute backend: the backend process can link against any host library (CUDA, ROCm, Metal, BLAS) without contaminating QEMU. VOGUE’s current evaluation uses direct QEMU `virtio-gpu-g1`; a `vhost-user-gpu` backend would be the correct isolation design for any future compute-remoting paper claim.

13.3 Guest Graphics Stacks

Linux DRM/KMS and libdrm. The Linux DRM subsystem provides the guest-side kernel driver (`virtio_gpu.ko`), GEM object management, KMS scanout, and the `libdrm` userspace API. It is the baseline guest stack for all VirGL and Venus evaluations on Linux. VOGUE explicitly replaces this stack with `libukdrm_compat` and `libukgbm_compat`, which expose the minimal API surface needed for `kmscube` and `glmark2` to compile and link without importing DRM ioctls, device files, or GEM. The compatibility boundary is intentionally narrow: API compatibility, not implementation correctness for the full DRM ABI.

Mesa, GBM, EGL, and GLES. Mesa [17] is the open-source userspace OpenGL/Vulkan stack on Linux. For VirtIO-GPU, Mesa provides the VirGL Gallium driver (guest-side command encoder), the Venus driver, the GBM/EGL window system integration layer, and software rasterizers (`llvmpipe`, `softpipe`, `OSMesa`). The VirGL Gallium driver inside Mesa is the reference implementation of the guest command encoder that VOGUE must eventually reimplement in `libukvirtio_gpu`. Rather than porting Mesa — its build system, LLVM dependency, and size are incompatible with Unikraft’s link-time elimination model — VOGUE implements its own narrower encoder targeting only the Gallium commands required for the target workloads. `libukegl` and the GBM shim serve the same function as Mesa’s EGL/GBM for the software-render path, bridging application code to the VOGUE display pipeline without Mesa’s broader dependency chain.

OpenGL and Vulkan. OpenGL [9] is the principal API exposed through the VirGL path. Vulkan [10], the modern explicit API, is exposed through Venus. For VOGUE’s current executable workloads, OpenGL ES 2.0 suffices (`kmscube`, `glmark2`), while Vulkan/Venus is tracked as a readiness-gated next stage rather than a completed runtime. For compute applications (LLMs), Vulkan compute shaders and SPIR-V provide a vendor-neutral path, but they require the full Venus protocol and Mesa Vulkan guest stack — a substantially larger undertaking than the VirGL/GLES2 path.

13.4 GPU Passthrough

VFIO and `vfio-pci`. VFIO [15] provides IOMMU-protected direct passthrough of physical PCIe devices to guest VMs. A guest with VFIO passthrough can run a full vendor GPU driver stack (CUDA, ROCm, Metal via translation) at near-native performance. However, VFIO passthrough requires

PCI enumeration, BAR mapping, MSI/MSI-X interrupt routing, and a complete vendor driver inside the guest. None of these components exist in a standard Unikraft build, and their introduction would consume tens of megabytes of code, violating unikernel minimality constraints. VFIO passthrough is the correct answer for performance-critical GPU workloads where isolation requirements permit it, but it is explicitly out of scope for VOGUE.

Confidential GPU. NVIDIA H100 and successor GPUs support a Confidential Compute mode [19] that establishes a protected channel between CPU and GPU TEE, allowing GPU computation to proceed without the host hypervisor observing GPU-resident data. This path requires hardware support, vendor-specific firmware, and a confidential VM setup incompatible with general-purpose VirtIO-GPU. Confidential GPU is the correct architecture for privacy-sensitive AI workloads (e.g., fine-tuning on regulated medical or financial data inside a CVM), but it is beyond VOGUE’s current scope. The interaction with VirtIO-GPU is also adversarial: VirtIO shared memory inherently exposes guest buffers to the host [23], weakening confidentiality guarantees that a confidential GPU is meant to provide.

13.5 LLM Compute Remoting

llama.cpp and GGML. `llama.cpp` and its underlying GGML tensor library [6] constitute the primary CPU-capable LLM inference stack in C/C++. GGML supports pluggable compute backends: CPU (reference), CUDA, Metal, Vulkan, and SYCL. Because GGML can run entirely on CPU with no GPU dependency, it is the most tractable LLM workload to run inside a Unikraft appliance today. The substrate evidence rows (`gfx.kmscube.sw`, `gfx.glmark2.sw`) validate the application framework on which an `app-uk11m` port would build.

GGML-VirtGPU. GGML-VirtGPU [7] is a `llama.cpp` backend that offloads GGML tensor operations to host GPU hardware via the VirtIO-GPU and `virglrenderer` API-remoting (Vulkan/Venus) path. The guest GGML backend packs tensor operation descriptors into `virglrenderer` API-remoting calls; the host-side renderer dispatches these to CUDA or the host GGML backend, returning results through shared memory. This design closely parallels VOGUE’s architecture for graphics: a minimal guest-side protocol library replaces the Linux DRM+Mesa guest stack. The critical difference is that GGML-VirtGPU assumes the Linux guest `virtio_gpu` DRM driver and `libdrm` ioctls; replacing that dependency with `libukvirtio_gpu` (backed by upstream `uksglist` scatter-gather) is the planned VOGUE extension. Those scatter-gather backing semantics directly map to the shared-memory buffer management that GGML-VirtGPU requires for zero-copy tensor sharing between guest and host.

Virglrenderer Vulkan/Venus. The API-remoting trampoline in `virglrenderer` provides the host-side dispatch mechanism for both GGML-VirtGPU and hypothetical fu-

Table 28: Implementation references for each VOGUE layer. The “Reference” column names the upstream spec or source file used; “What It Specifies” describes the specific mechanism VOGUE depends on.

VOGUE Layer	Reference	What It Specifies
libukvirtio_gpu — 2D commands	OASIS VirtIO Specification v1.3, §7.8 “virtio-gpu device”	Resource lifecycle commands (RESOURCE_CREATE_2D, ATTACH_BACKING, TRANSFER_TO_HOST_2D, SET_SCANOUT, RESOURCE_FLUSH), fence semantics, feature bits
libukvirtio_gpu — 3D / blob	VirtIO-GPU 3D extension; virtio_gpu_proto.h in Linux kernel tree include/uapi/linux/virtio_gpu.h	CTX_CREATE, SUBMIT_3D, RESOURCE_CREATE_3D, RESOURCE_CREATE_BLOB, RESOURCE_MAP_BLOB, RESOURCE_UNMAP_BLOB, UUID; feature bits VIRTIO_GPU_F_VIRGL, VIRTIO_GPU_F_RESOURCE_BLOB, VIRTIO_GPU_F_CONTEXT_INIT, VIRTIO_GPU_F_BLOB_ALIGNMENT (bit 5); virtio_gpu_config is now 20 bytes (5 fields) including blob_alignment at offset 16
libukvenus — Venus ring protocol	Mesa source src/virtio/vulkan/vn_ring.h and vn_ring.c; KhronosGroup/venus-protocol include/vn_protocol_driver_defines.h	Ring head/tail/status layout at offsets 0/64/128/192; circular write with wrap-around; vkCreateRingMESA / vkNotifyRingMESA / vkDestroyRingMESA entry points; packed struct layout with no inter-field padding
libukvenus — Venus command encoding	Mesa src/virtio/vulkan/vn_protocol_driver_*.h; vn_encode_array_size() in vn_cs.h	Array presence encoded as uint32_t n; zero = absent; all structs packed; command IDs from vn_protocol_driver_defines.h
libukggml_vulkan — static Vulkan dispatch	Vulkan 1.3 specification §3.2 “Command Function Pointers”; Vulkan-Headers v1.3.352 vulkan/vulkan.h	80+ Vulkan entry points; vkGetDeviceProcAddr dispatch table; VULKAN_HPP_DEFAULT_DISPATCHER initialization pattern
apps/app-llama-upstream* — in-tree ggml	llama.cpp docs/build.md; Unikraft APG porting guide; upstream SHA fcae601e4	In-tree source compilation pattern; Makefile.uk addlib_s + per-file SRC-C/SRC-CXX; upstream lib-musl supplies mmap/sysconf/getauxval/prctl/pthread_setaffinity_np
apps/app-llama-upstream-vk — Vulkan ggml	ggml-vulkan backend docs/backend/VULKAN.md; SPIR-V headers (KhronosGroup/SPIRV-Headers)	GGML_USE_VULKAN=1; uk_ggml_vk_loader.cpp wires VULKAN_HPP_DEFAULT_DISPATCHER; SPIR-V shader table requires SPIR-V headers at build time
kraft/Kraftfile.llama-upstream* — 9pfs mount	Unikraft lib-9pfs documentation; CONFIG_LIBVFSSCORE_AUTOMOUNT_CI_RAMFS; VirtIO-9P QEMU device (-virtfs)	CONFIG_LIBRAMFS=y, CONFIG_LIBVFSSCORE_AUTOMOUNT_CI=y, CONFIG_LIBVFSSCORE_AUTOMOUNT_CI_RAMFS=y required before mkdir/mount; CONFIG_LIBUK9P=y, CONFIG_LIBVIRTIO_9P=y, CONFIG_LIB9PFS=y for 9pfs transport
QEMU virtio-gpu-gl-pci,venus=true	QEMU 11.0 release notes; QEMU docs/system/devices/virtio-gpu.rst	-device virtio-gpu-gl-pci,hostmem=512M,blob=true,venus=true enables Venus capset id=4; -display egl-headless,gl=on (or another GL-capable backend) is required by QEMU virtio-gpu-gl before Venus initializes
virglrenderer Venus decoder	virglrenderer source src/venus/; vkr_context_submit_cmd()	Decodes SUBMIT_3D payload as Venus commands; expects packed layout; context initialized via DRM_IOCTL_VIRTPGPU_CONTEXT_INIT with capset id=4

ture Unikraft compute offload. The guest issues opaque context commands (SUBMIT_3D with Vulkan/Venus-encoded payloads); virglrenderer deserializes and dispatches them to registered backends. This mechanism allows a single VirtIO-GPU frontend to serve multiple use cases — graphics, compute, ML inference — purely by negotiating different capsets and command encodings. VOGUE’s libukvirtio_gpu already implements the transport-level infrastructure (CTX_CREATE, SUBMIT_3D wire format); the remaining work is encoding the Vulkan/Venus command payload correctly.

vAccel. vAccel [29] is a modular hardware acceleration framework designed to expose heterogeneous accelerators

(GPU, FPGA, NPU) to virtualized and containerized workloads through a unified API. Rather than remoting a specific GPU API, vAccel defines its own higher-level operations (image classification, object detection, generic tensor compute) and provides backend plugins for CUDA, TensorFlow, TensorRT, and others. VOGUE and vAccel occupy adjacent design points: VOGUE implements the low-level VirtIO-GPU transport for graphics and compute, while vAccel abstracts over multiple accelerator types at a higher semantic level. A future integration could route vAccel operations through the VirtIO-GPU Vulkan/Venus path, using libukvirtio_gpu as the transport.

Cricket. Cricket [13] is a CUDA virtualization layer providing transparent remote execution and checkpoint-restart for CUDA applications. It intercepts CUDA driver API calls, serializes them, and dispatches to a remote GPU over a network transport. Cricket is CUDA-specific and much heavier than VirtIO-GPU Vulkan/Venus remoting: it requires a full CUDA runtime on both ends, whereas VirtIO-GPU Vulkan/Venus operates at a transport level that is API-agnostic. Cricket is relevant primarily as a performance baseline for GPU API remoting latency and bandwidth benchmarks.

vLLM, SGLang, and TensorRT-LLM. Production LLM serving stacks such as vLLM, SGLang, and TensorRT-LLM implement advanced schedulers, continuous batching, KV-cache management, and quantized inference against CUDA or ROCm. These are host-side systems that VOGUE does not attempt to port; they serve as external performance baselines for throughput and latency under the same LLM workloads that GGML-VirtGPU targets. The relevant comparison axis is: what fraction of host GPU throughput is preserved after the VirtIO-GPU Vulkan/Venus remoting overhead? That question cannot be answered without a working GGML-VirtGPU + libukvirtio_gpu stack, which makes it K1-equivalent future work.

13.6 Summary and Position of VOGUE

13.7 Implementation References

This section records the upstream specifications, header files, and documentation used to implement each VOGUE layer. These are the authoritative sources a future implementer should consult to reproduce or extend the work.

The central dependency for K1 accelerated rendering is now satisfied on the evaluation host: xport.qemu-vgpu, Venus bootstrap command serialization, the Mesa ring-buffer protocol, and the KMSCube submit/frame rows all have current same-run evidence. The architecture VOGUE has built for the 2D path (scatter-gather backing via upstream uksglist, protocol command staging, fence synchronization in libukvirtio_gpu) is the same architecture required for 3D; future work is broader app coverage and performance rather than first transport proof.